DATA STRUCTURES & ALGORITHMS • 游戏客户端秋招版

排序算法：从原理到工程

十种经典排序的思想、伪代码、可视化与 C++ 实现；std::sort 内部到底干了什么；以及排序在渲染队列、UI 层级、帧同步、排行榜里的真实用法和坑。

10 种算法 + 3 种快排变体　4 个交互实验　MSVC /O2 实测 10 组实验　40 道面试追问

CHAPTER 00

先背这个：30 秒面试版答案

面试官问“讲讲你知道的排序算法”，先用一分钟给出全景，再等对方挑一个深挖。

问：讲讲常见的排序算法，以及你会怎么选？

按思想分三类。

- **O(n²) 的简单排序**：冒泡、选择、插入。插入排序最有价值，它的代价是 O(n + 逆序对数)，对小数组和近乎有序的数据很快，所以工业级排序都拿它处理 16 ~ 32 个元素以内的小区间。
- **O(n log n) 的比较排序**：归并（稳定，最坏也是 n log n，但要 O(n) 额外空间）、快排（平均最快、原地，但最坏 O(n²)，而且不稳定）、堆排（最坏 n log n、O(1) 空间，但缓存不友好，常数大）。**任何比较排序最坏都至少要 Ω(n log n) 次比较**，这个下界可以用决策树证明。
- **非比较排序**：计数、基数、桶。它们利用键本身的结构，可以做到 O(n)，但对键的类型和值域有要求。

工程上，`std::sort` 是 introsort：快排做主体，递归太深就切到堆排兜底最坏情况，小区间交给插入排序。它不稳定；要稳定就用 `std::stable_sort`（归并），只要前 K 个就用 `partial_sort` / `nth_element`。

放到游戏客户端：渲染队列用 64 位 sort key 做基数排序；UI 层级必须稳定；帧同步里比较器必须是带 id 兜底的全序，否则各端平局顺序不一样，就会不同步；比较器一定要满足严格弱序，写成 `≤` 会让某些 STL 实现越界写内存。

全家福速查表

算法	最好	平均	最坏	额外空间	稳定	一句话特点
冒泡	O(n)	O(n²)	O(n²)	O(1)	✓	相邻交换；加“无交换就退出”后最好 O(n)
选择	O(n²)	O(n²)	O(n²)	O(1)	✗	交换次数最少（≤ n−1），比较次数永远 n²/2
插入	O(n)	O(n²)	O(n²)	O(1)	✓	代价 = O(n + 逆序对)；小数组 / 近乎有序时最快
希尔	O(n log n)	≈O(n^1.3)	依增量序列，O(n^1.5) 或 O(n^4/3)	O(1)	✗	分组插入，大步长先粗调
归并	O(n log n)	O(n log n)	O(n log n)	O(n)	✓	稳定 + 最坏有保证；链表 / 外部排序首选
快排	O(n log n)	O(n log n)	O(n²)	O(log n) 栈	✗	平均最快；pivot 选不好会退化
堆排	O(n log n)	O(n log n)	O(n log n)	O(1)	✗	最坏有保证且原地；introsort 的兜底
计数	O(n+k)	O(n+k)	O(n+k)	O(n+k)	✓	值域 k 小的整数
基数	O(d(n+b))	O(d(n+b))	O(d(n+b))	O(n+b)	✓	定长整数键；渲染 sort key 常用
桶	O(n)	O(n+k)	O(n²)	O(n+k)	**取决于桶内**	数据分布均匀时接近 O(n)

面试注意：最容易丢分的五句话

- “快排空间复杂度 O(1)” —— **错。**递归栈平均 O(log n)，朴素实现最坏 O(n)；只有“递归短边、循环长边”才能保证 O(log n)。
- “堆排序比快排快，因为最坏也是 n log n” —— **错。**实测堆排更慢（本书 E5：1e6 随机 int，堆 73.6 ms，工程快排 47.3 ms），原因是缓存不友好、比较次数大约多一倍。
- “std::sort 就是快排” —— **不完整。**它是 introsort（快排 + 堆排 + 插排），从 C++11 起标准要求最坏 O(n log n)。
- “插入排序没用” —— **错。**所有工业排序都拿它处理小区间；帧间几乎不变的数据，用它增量重排最快（E8：0.005 ms，std::sort 0.062 ms）。
- “比较器返回 a <= b 没问题” —— **错。**这违反严格弱序，属于未定义行为（E7：libstdc++ 式的无哨兵插入越界写了 7 个格子）。

本书怎么读

第一部分 · 度量衡

稳定性、逆序对、n log n 下界、选型决策树。先建立判断标准。

第二 ~ 四部分 · 十种算法

每种算法按同样的顺序讲：思想 → 图解 → 伪代码 → C++ → 复杂度 → 交互实验 → 面试点。

第五部分 · 工业实现

std::sort / stable_sort / partial_sort / nth_element 的内部，严格弱序，TimSort / pdqsort。

第六部分 · 游戏实战

渲染队列、透明排序、UI 层级、帧同步确定性、Top-K、Sweep and Prune、小栈线程。

关于实测数据

书中所有“实测”都来自同目录 `demos/sort_demo.cpp`，在 Windows 11 + MSVC 14.44（`cl /std:c++latest /EHsc /O2`，x64）下运行，完整输出见 `demos/run_output.txt`。共 10 组实验，编号 E1 ~ E10，正文引用时标出编号。绝对耗时和机器有关，**请看相对关系。**

PART ONE

第一部分　度量衡：怎么评价一个排序算法

只背“时间复杂度”是不够的。稳定性、额外空间、对已有序数据是否更快、缓存是否友好、比较器有什么要求，这些才是工程里真正决定选型的因素。

CHAPTER 01

排序的六个维度与 n log n 下界

先把评价标准讲清楚，后面每讲一种算法，都从这几个维度去打分。

本章目标

- 说清稳定、原地、自适应、比较排序 / 非比较排序这几个词的准确含义
- 理解逆序对：它是冒泡和插入排序的真实工作量
- 能用决策树证明比较排序的下界 Ω(n log n)
- 拿到一个场景，能用一棵决策树选出合适的排序

1.1 六个评价维度

维度	含义	为什么在游戏客户端里要关心
时间复杂度	最好 / 平均 / 最坏三种情况分开看	每帧的预算只有 16.6 ms（60 FPS），偶发的最坏情况就是一次卡顿
额外空间	原地（O(1) 或 O(log n) 栈）还是要 O(n) 缓冲	移动端内存紧；每帧临时分配会带来碎片和分配开销
稳定性	键相等的元素，排序后是否保持原来的相对顺序	UI 同层节点的先后、多关键字排序、帧同步的确定性
自适应性	输入越接近有序，是否跑得越快	渲染队列、碰撞检测的端点，帧与帧之间几乎不变
比较 / 非比较	只靠 `a < b` 判断，还是利用键本身的结构（数位、值域）	64 位 sort key 可以用基数排序，突破 n log n
访存模式	顺序访问还是大跨度跳跃；交换的是小键还是大结构体	缓存未命中往往比比较次数更决定实际耗时

深入一层：“原地”的两种口径

严格说，原地（in-place）指额外空间 O(1)。但多数教材和面试默认把“只用 O(log n) 递归栈”的快排也算作原地。回答时可以主动区分：**“快排是原地的，但递归栈平均 O(log n)、最坏 O(n)；堆排序才是真正的 O(1)。”**这样说既严谨，又显得基础扎实。

1.2 稳定性：最常被追问的概念

稳定指：如果 `a[i]` 和 `a[j]` 的键相等，并且排序前 i < j，那么排序后 `a[i]` 仍然排在 `a[j]` 前面。只对整数本身排序时稳定性看不出来，它只在“按某个字段给记录排序”时才有意义。

交互实验 • 稳定性：同点数的牌，花色顺序还在吗？

原始顺序（同点数下：先出现的在前）：



插入排序　冒泡排序　归并排序　计数排序　选择排序　堆排序

快排 · Hoare + 三数取中　快排 · 三路划分



选择排序：不稳定（2♠ 跑到了 2♥ 前面；3♥ 跑到了 3♠ 前面；4♦ 跑到了 4♠ 前面）

稳定性有什么用？最典型的是**多关键字排序**：要“先按段位、段位相同再按胜率”排序，可以先按胜率排一次，再用**稳定**排序按段位排一次。第二次排序不会打乱段位相同的玩家已有的胜率顺序。基数排序就是靠这个原理工作的：从低位到高位，每一趟都必须稳定。

判断一个算法是否稳定，有一个简单的经验：**只做相邻交换，或者遇到相等就停下来不越过，就是稳定的；只要有“远距离交换”，通常就不稳定。**选择排序、堆排序、快排都会把元素一次换出很远，所以都不稳定。

实测 E1：5000 条记录、键只有 50 种，排序后统计“相邻同键但 id 逆序”的对数

冒泡 / 插入 / 归并 / 计数 / 基数 / `stable_sort` 都是 **0**；选择 1607，希尔 2503，工程快排 2498，三路快排 2454，堆 2916，`std::sort` 2509。

1.3 逆序对：冒泡和插入的真实工作量

如果 i < j 且 `a[i] > a[j]`，就称 (i, j) 是一个**逆序对**。比如 `[3, 1, 2]` 有 (3,1)、(3,2) 两个逆序对。逆序对数记作 I，有几个很有用的性质：

- 有序数组 I = 0；逆序数组 I = n(n−1)/2，这是最大值；随机排列的期望 I = n(n−1)/4。
- **交换一对相邻的逆序元素，I 恰好减 1。**所以冒泡排序的交换次数、插入排序的移动次数都正好等于 I。
- 由此可以得出：任何“只交换相邻元素”的排序，平均都要 $\Omega(n^2)$ 步。想突破 n^2，就必须做远距离交换，比如希尔、快排、堆排。

实测 E2：N=1000 的随机数据

冒泡写入 483422 次，等于交换 241711 次，也就是 I；插入比较 242702 次，约等于 I + n。两者都接近理论期望 n(n−1)/4 = 249750。数组逆序时，冒泡和插入的比较次数都是 499500 = n(n−1)/2。

1.4 比较排序的下界：为什么快不过 n log n

只能通过 `a < b ?` 获取信息的排序算法，可以抽象成一棵**决策树**：每个内部节点是一次比较，每个叶子是一种最终排列。n 个元素有 n! 种可能的排列，算法必须能区分所有排列，所以叶子至少有 n! 个。

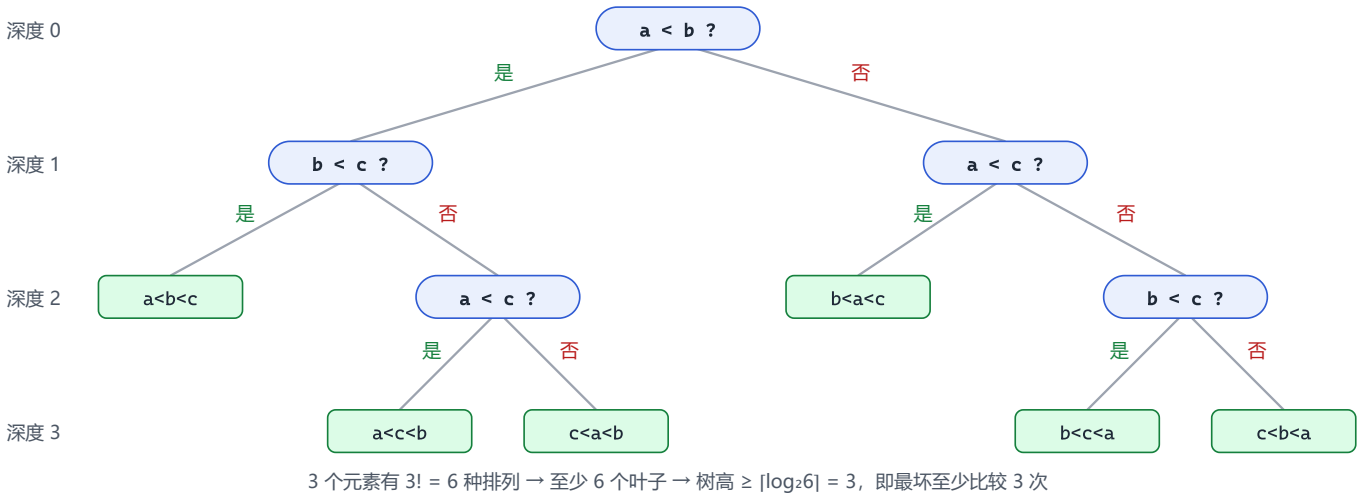


图 1-1　3 个元素的决策树：每条从根到叶的路径就是一次排序过程中做的全部比较

高度为 h 的二叉树最多有 2^h 个叶子，所以 $2^h \geq n!$，即 $h \geq \log_2(n!)$。由斯特林公式，$\log_2(n!) = n\log_2 n - n\log_2 e + O(\log n) \approx n\log_2 n - 1.44n$。所以**任何比较排序在最坏情况下都至少需要 $\Omega(n \log n)$ 次比较**。这个结论对平均情况同样成立：叶子的平均深度也至少是 $\log_2(n!)$。

面试速答版

“比较排序可以看成一棵决策树。它要区分 n! 种排列，所以至少有 n! 个叶子，树高就至少是 log(n!)，也就是 n log n 级别。计数排序、基数排序不在这个模型里，因为它们不靠比较，而是直接把键当下标用，所以能做到线性。”

1.5 增长曲线：n^2 和 $n \log n$ 差多少

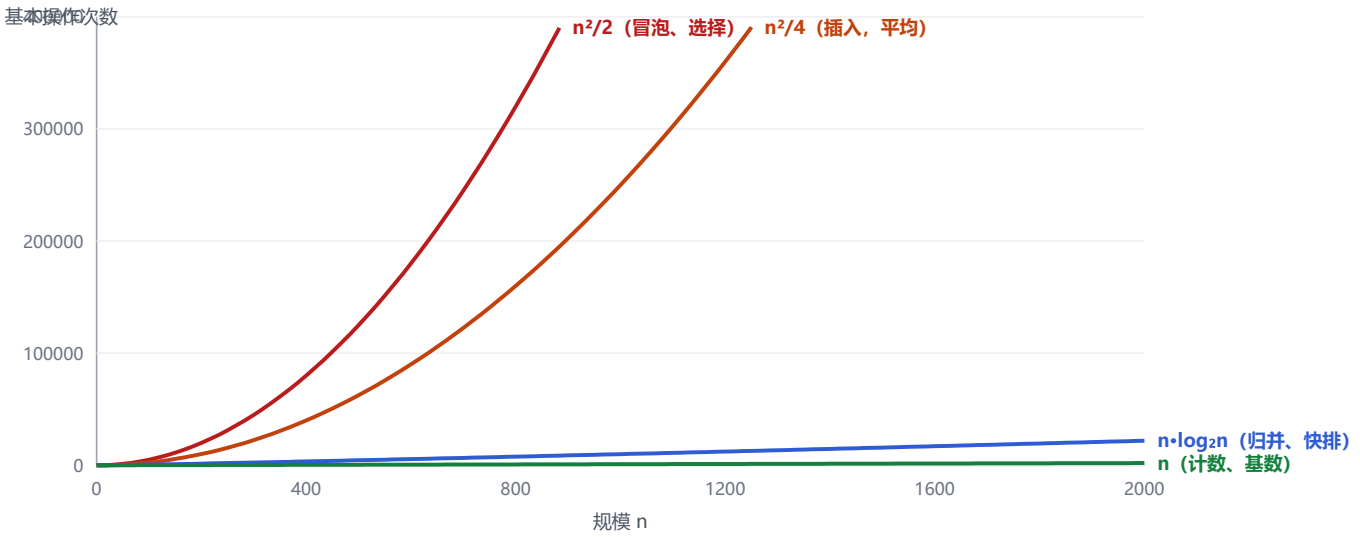


图 1-2　n=2000 时，$n^2/2$ = 200 万，$n \log_2 n \approx 2.2$ 万，相差约 90 倍；到 n=100 万时相差约 2.5 万倍

小 n 的时候常数因子更重要。n = 16 时，$n^2/4 = 64$，$n \log_2 n = 64$，两者已经持平；再加上插入排序没有递归、访存是顺序的，所以“小区间用插入排序”是所有工业排序的共识。libstdc++ 的阈值是 16，MSVC 是 32。

1.6 选型决策树

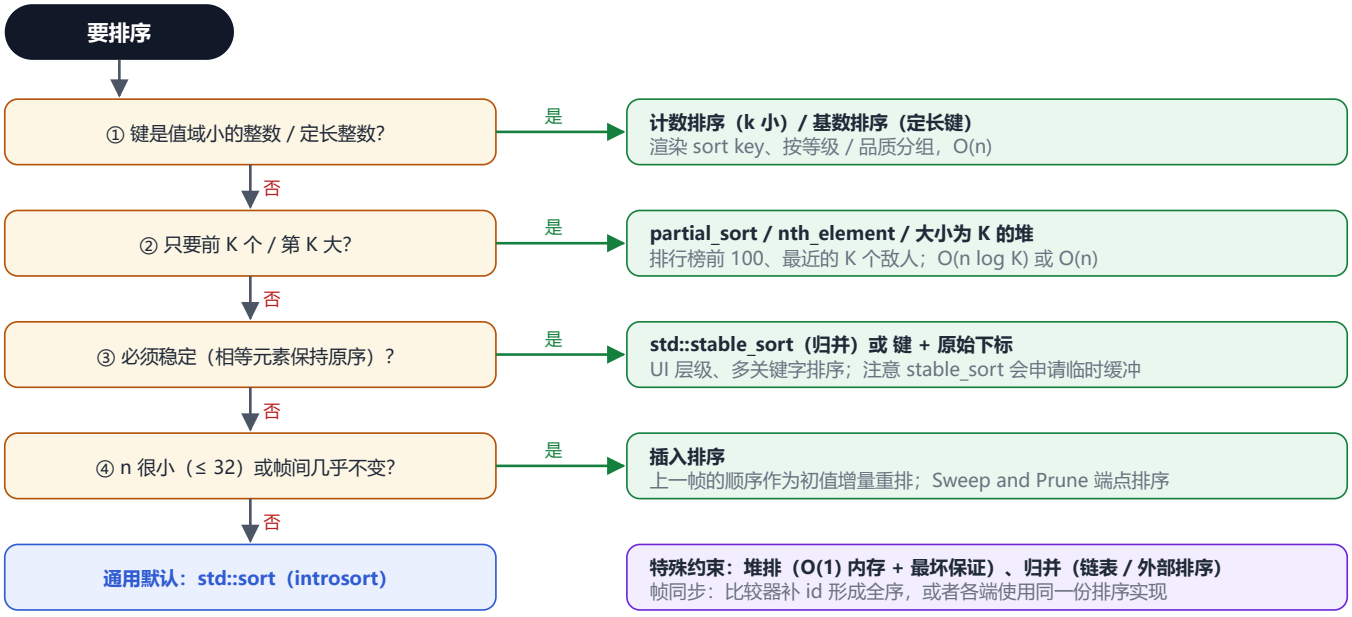


图 1-3　选型决策树：从上往下问四个问题，大多数场景都能落到一个确定的选择

本章速记

- 六个维度：**时间（最好 / 平均 / 最坏）、空间、稳定、自适应、比较与否、访存模式。**
- 稳定的经验判断：只做相邻交换、遇到相等就停 → 稳定；有远距离交换 → 一般不稳定。
- 冒泡的交换次数、插入的移动次数都等于**逆序对数**；只做相邻交换的排序平均 $\Omega(n^2)$。
- 比较排序下界：决策树至少有 n! 个叶子，树高 $\geq \log_2 n! \approx n \log_2 n - 1.44n$。

▶ **自测：为什么说“只做相邻交换的排序平均是 $\Omega(n^2)$”？**

▶ **自测：计数排序是 O(n)，和 Ω(n log n) 下界矛盾吗?**

PART TWO

第二部分　O(n²) 三兄弟与希尔排序

冒泡、选择、插入是所有排序的起点。它们在大数据上没有竞争力，但其中的想法（相邻交换、找最值、有序区扩张）在每一个工业排序里都能找到。

CHAPTER 02

冒泡排序与选择排序

一个靠相邻交换让最大值“浮”到末尾，一个每轮直接找出最小值换到前面。两者都是 O(n²)，但性格完全不同。

本章目标

- 写出带两种优化的冒泡排序（无交换提前退出、记录最后交换位置）
- 举出选择排序不稳定的反例，说清它“交换次数最少”的意义
- 理解为什么同样是 O(n²)，实测速度可以差好几倍

2.1 冒泡排序：相邻比较，大的往后沉

从头到尾比较每一对相邻元素，前一个比后一个大就交换。这样一趟下来，**当前范围内最大的元素一定会被“推”到最后**，就像气泡上浮一样。每趟结束后有序区增加一个元素，n−1 趟后全部有序。



图 2-1　冒泡排序：第一趟让最大的 8 沉到末尾；第 3 趟没有发生交换，于是提前结束

伪代码

冒泡排序 · 伪代码

```
bubbleSort(a, n):
    end = n - 1
    while end > 0:
        last = 0                        // 本趟最后一次交换的位置
        for j = 0 to end - 1:
            if a[j] > a[j+1]:           // 严格大于才交换 → 稳定
                swap(a[j], a[j+1])
                last = j
        end = last                      // last 之后的部分已经有序；本趟无交换时 last = 0，循环结束
```

C++ 实现

bubble_sort.h

```cpp
template <class T, class Less = std::less<T>>
void bubble_sort(T* a, int n, Less less = {}) {
    int end = n - 1;
    while (end > 0) {
        int last = 0;
        for (int j = 0; j < end; ++j) {
            if (less(a[j + 1], a[j])) {         // 只用 <，保证相等不交换
                std::swap(a[j], a[j + 1]);
                last = j;
            }
        }
        end = last;    // 优化 1：没交换就退出；优化 2：[last+1, end] 已有序，下趟不用再扫
    }
}
```

深入一层：两种优化各解决什么问题

- **提前退出**：一趟没有交换说明已经有序。它让最好情况（输入有序）从 $O(n^2)$ 降到 $O(n)$。实测 E3 中，有序的 2 万个数只要 0.01 ms，随机的要 75 ms。
- **记录最后交换位置**：如果数组尾部本来就有序，一趟就能把这段整体划进有序区，不用每趟只前进一格。
- 两种优化都改变不了一个弱点：**小元素在末尾时，每趟只能向前挪一格**（俗称“乌龟”）。比如 `[2,3,4,5,1]` 需要 4 趟。**鸡尾酒排序**（双向冒泡）交替从左往右、从右往左扫，能缓解这个问题，但仍然是 $O(n^2)$。

复杂度与性质

情况	比较次数	交换次数	说明
最好（已有序）	n − 1	0	依赖“提前退出”优化
平均（随机）	≈ $n^2/2$	= 逆序对数 ≈ $n^2/4$	实测 E2（n=1000）：比较 496260，交换 241711
最坏（逆序）	$n(n-1)/2$	$n(n-1)/2$	实测 E2：比较 499500，写 999000

稳定：只交换相邻元素，而且严格大于才交换，相等的元素不会互相越过。**原地**：$O(1)$ 额外空间。

2.2 选择排序：每轮找最小，换到最前面

把数组分成左边的有序区和右边的无序区。每一轮在无序区里**扫描一遍找出最小值**，和无序区的第一个元素交换，有序区就增加一个元素。

它的问题出在“交换”这一步：被换走的那个元素可能一下跳过很远，越过了和它相等的元素。下面这个例子里，`5a` 和 `5b` 的键都是 5，排序前 5a 在前面：



图 2-2　选择排序的不稳定反例：第一次交换就把 5a 远远甩到了 5b 之后

selection_sort.h

```
template <class T, class Less = std::less<T>>
void selection_sort(T* a, int n, Less less = {}) {
    for (int i = 0; i < n - 1; ++i) {
        int m = i;
        for (int j = i + 1; j < n; ++j)
            if (less(a[j], a[m])) m = j;        // 只记下标，不交换
        if (m != i) std::swap(a[i], a[m]);      // 每轮最多交换一次
    }
}
```

复杂度与性质

- **比较次数永远是 n(n−1)/2**，和输入无关。所以它**不是自适应的**，有序数组也要跑满 $O(n^2)$。实测 E3 中，有序和随机都约 280 ms。
- **交换次数 ≤ n−1**，这是所有算法里最少的一类（E2：n=1000 随机，只写了 1982 次，而冒泡写了 48 万次）。当“写”远比“读”昂贵时（比如 Flash 存储），它反而有优势。如果要让写次数绝对最少，可以用 Cycle Sort。
- **不稳定**（见图 2-2）。如果非要稳定，可以把“交换”改成“把最小值插到前面、中间整体后移”，代价是每轮 O(n) 次移动；或者在链表上实现。

变体：双向选择

每轮同时找出最小值和最大值，分别放到两端，扫描轮数减半，但比较次数基本不变（每个元素仍要和 min、max 各比一次）。面试里提一下就够了，它不会改变复杂度的量级。

2.3 同样是 O(n²)，为什么实测差好几倍



图 2-3 三种 O(n²) 算法之间也差了 14 倍，而 O(n²) 和 O(n log n) 之间差了两到三个数量级

复杂度只描述增长的趋势，不包含常数。三兄弟的比较次数差不多（选择 n²/2，冒泡约 n²/2，插入约 n²/4），但每次比较附带的工作差别很大：

- **插入排序**：平均只比较 n²/4 次，每次后移只有一次赋值，内层循环遇到不大于 key 的元素就停。所以它在三兄弟里最快。
- **冒泡排序**：一次交换要三次赋值，而且随机数据下 `if` 分支几乎没法预测。
- **选择排序**：`m = j` 让下一次比较的 `a[m]` 依赖上一次的结果。如果编译器把它生成成条件传送（cmov），就会形成一条循环携带的依赖链，CPU 的乱序执行没法把这些比较并行起来。这一点要谨慎看待：同一台机器上另一种写法（`demos/sel_test.cpp`）只要 38 ms。这说明**O(n²) 算法的常数对代码生成非常敏感**，比较次数却是固定的。

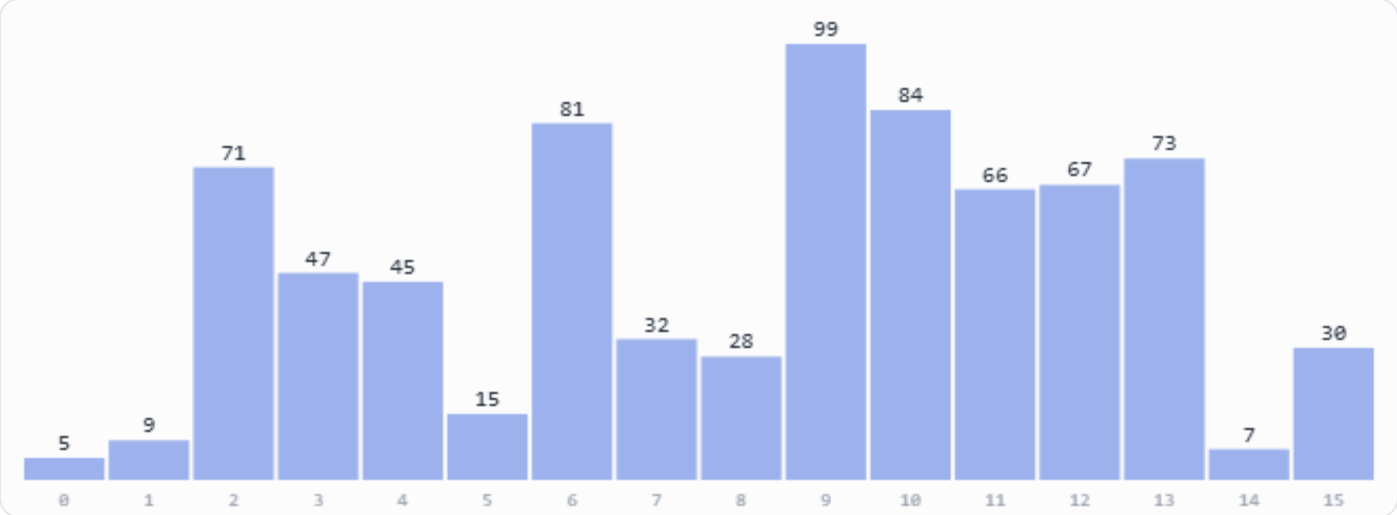
面试注意

被问“冒泡和插入哪个快”，答“插入更快”，然后给出两个理由：①平均比较次数少一半（n²/4 对 n²/2）；②每次移动只有一次赋值，冒泡的一次交换是三次赋值。如果能补一句“二者的交换 / 移动次数都等于逆序对数”，就说明你理解了本质。

2.4 交互实验

切到“已有序”，看冒泡怎么一趟就结束；切到“逆序”，看选择排序的比较次数为什么和随机数据完全一样。

交互实验 • 排序过程可视化




```
for end = n-1 downto 1:
  swapped = false
  for j = 0 to end-1:
    if a[j] > a[j+1]:
      swap(a[j], a[j+1]); swapped = true
  // 本趟最大值已沉到 a[end]
  if not swapped: break     // 提前结束
```

点击「播放」或「单步」开始。

比较/读取 交换 写入 pivot 当前元素 已就位 不在当前子区间

本章速记

- 冒泡：相邻交换、**稳定**；“提前退出”让最好情况变成 O(n)；交换次数 = 逆序对数。
- 选择：比较次数固定 $n^2/2$，**不自适应**；交换 ≤ n−1 次；远距离交换导致**不稳定**。
- 同为 $O(n^2)$，实测插入 < 冒泡 < 选择，常数差距可达十几倍。

▶ **自测：冒泡排序最好情况是 O(n) 吗?**

▶ **自测：选择排序可以做成稳定的吗?**

CHAPTER 03

插入排序与希尔排序

插入排序是 O(n²) 里唯一会被工业代码天天调用的算法。希尔排序则说明，只要允许远距离移动，就能打破“相邻交换必然 n²”的限制。

本章目标

- 写出移位版插入排序，说清它为什么稳定、自适应、在线
- 理解“代价 = O(n + 逆序对)”，以及工业排序为什么都用它处理小区间
- 理解希尔排序的分组思想和增量序列对复杂度的影响

3.1 插入排序：像整理手里的扑克牌

左边是已经排好的牌，每次从右边摸一张（key），从右往左和有序区比较：比 key 大的牌往右挪一格，直到遇到不大于 key 的牌，把 key 插到它后面。



图 3-1 插入排序：每个元素后移的次数，正好等于它和左边元素构成的逆序对数

插入排序 · 伪代码

```
insertionSort(a, n):
    for i = 1 to n-1:
        key = a[i]
        j = i - 1
        while j ≥ 0 and a[j] > key:    // 严格大于才后移 → 相等元素不越过 → 稳定
            a[j+1] = a[j]
            j = j - 1
        a[j+1] = key
```

insertion_sort.h（移位版，比“相邻交换版”少 2/3 的赋值）

```cpp
template <class T, class Less = std::less<T>>
void insertion_sort(T* a, int n, Less less = {}) {
    for (int i = 1; i < n; ++i) {
        T key = std::move(a[i]);
        int j = i - 1;
        while (j ≥ 0 && less(key, a[j])) {   // key < a[j] 才后移
            a[j + 1] = std::move(a[j]);
            --j;
        }
        a[j + 1] = std::move(key);
    }
}
```

复杂度：O(n + I)

每个元素后移的次数，等于它左边比它大的元素个数。把所有元素加起来，总移动次数恰好是逆序对数 I，比较次数在 I 到 I + n − 1 之间。所以：

输入	逆序对 I	时间	实测 E2（n=1000）比较次数	实测 E3（n=20000）
已有序	0	O(n)	999	0.01 ms
近乎有序	O(n)	O(n)	—	0.57 ms
随机	≈ $n^2/4$	$O(n^2)$	242702	20.57 ms
逆序	$n(n-1)/2$	$O(n^2)$	499500	40.00 ms

它同时具备四个好性质：**稳定**、**原地**、**自适应**（越有序越快）、**在线**（数据一个个到达时可以边收边排）。

3.2 为什么工业排序都离不开它

- **小数组上常数最小**：没有递归、没有额外内存、访存是连续的。16 ~ 32 个元素正好在一两条缓存行里。libstdc++ 的 `std::sort` 在区间 ≤ 16 时转插入排序，MSVC 的阈值是 32，TimSort 用二分插入排序把短段补齐到 minrun（32 ~ 64）。
- **收尾快**：快排把大区间切到“块内无序、块间有序”之后，整体只剩很少的逆序对，最后统一做一遍插入排序就是线性的。libstdc++ 的 `__final_insertion_sort` 就是这个思路。
- **帧间增量**：游戏里很多数据上一帧已经排好，这一帧只有少数元素变化，比如渲染物体的深度、碰撞检测的 AABB 端点。直接拿上一帧的顺序做插入排序几乎是 O(n)。

实测 E8：帧间增量重排（N=20000，上一帧已排好，本帧 2% 的 key 发生微小变化）

插入排序 **0.005 ms**，`std::sort` 0.062 ms，基数排序 0.094 ms。插入排序快了 12 倍以上，因为它几乎只是线性扫了一遍。

两个经典优化

① **哨兵（无边界检查）**：先把全局最小值换到 `a[0]`，内层循环就不用再判断 `j ≥ 0`，因为 key 一定会在 `a[0]` 处停下。libstdc++ 的 `__unguarded_linear_insert` 就是这么做的。这个优化的前提是**比较器满足严格弱序**，否则 key 会越过 a[0] 继续往左走，读写数组外的内存（第 8 章 E7 会实测）。

② **二分插入**：在有序区里用二分查找插入位置，比较次数降到 O(n log n)，但移动次数仍然是 O(n^2)。比较代价高（比如比较字符串）时有用。注意要用 `upper_bound`，才能保持稳定：

二分插入排序

```cpp
template <class T, class Less = std::less<T>>
void binary_insertion_sort(T* a, int n, Less less = {}) {
    for (int i = 1; i < n; ++i) {
        T* pos = std::upper_bound(a, a + i, a[i], less);    // 第一个 > key 的位置：相等的留在左边
        T key = std::move(a[i]);
        std::move_backward(pos, a + i, a + i + 1);          // [pos, i) 整体右移一格
        *pos = std::move(key);
    }
}
```

3.3 希尔排序：先远后近的插入排序

插入排序慢在每次只能挪一格。希尔排序（Donald Shell，1959）的想法是：先取一个较大的步长 gap，把**相距 gap 的元素看成一组**，各组分别做插入排序，让大元素一步跳很远。然后逐渐缩小 gap，最后 gap = 1 时做一次普通插入排序。这时数组已经基本有序，最后一步代价很低。



图 3-2　希尔排序（gap = 5, 2, 1）：大步长先把元素送到大致的位置，最后一轮插入排序几乎不用移动

shell_sort.h（Ciura 增量序列）

```
template <class T, class Less = std::less<T>>
void shell_sort(T* a, int n, Less less = {}) {
    std::vector<int> gaps = {1, 4, 10, 23, 57, 132, 301, 701, 1750};   // Ciura 2001，实验得到
    while (gaps.back() < n / 2) gaps.push_back(int(gaps.back() * 2.25)); // 更大的 n 按 ×2.25
    for (int g = int(gaps.size()) - 1; g ≥ 0; --g) {
        const int gap = gaps[g];
        for (int i = gap; i < n; ++i) {              // 所有组交替推进，不必一组一组地做
            T key = std::move(a[i]);
            int j = i;
            while (j ≥ gap && less(key, a[j - gap])) {
                a[j] = std::move(a[j - gap]);
                j -= gap;
            }
            a[j] = std::move(key);
        }
    }
}
```

增量序列决定复杂度

序列	取值	最坏复杂度	备注
Shell 原始	n/2, n/4, …, 1	$O(n^2)$	奇偶位置在 gap=1 之前从不比较，容易被构造出最坏输入
Hibbard	1, 3, 7, 15, …（2^k-1）	$O(n^{1.5})$	相邻增量互质
Knuth	1, 4, 13, 40, …（$(3^k-1)/2$）	$O(n^{1.5})$	好记，常被用作面试答案
Sedgewick	1, 5, 19, 41, 109, …	$O(n^{4/3})$	理论上较优
Ciura	1, 4, 10, 23, 57, 132, 301, 701	未知	实验得到的最佳序列，本书 demo 使用

希尔排序**不稳定**（不同组之间会互相越过，E1 实测逆序对 2503），**原地**，平均复杂度目前没有精确的理论结果，实践中大约是 $O(n^{1.25})$ ~ $O(n^{1.3})$。

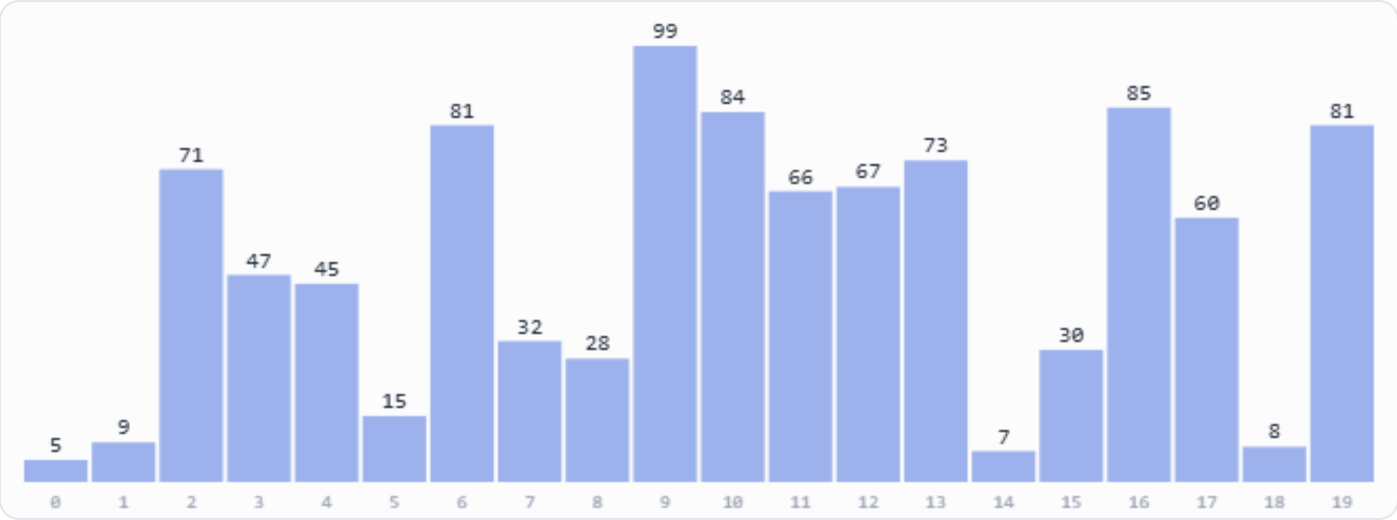
实测 E5：N = 100 万随机 int

希尔 87.67 ms，`std::sort` 51.38 ms，只慢了 1.7 倍，比堆排序（73.59 ms）也差不了多少。它代码短、无递归、无额外内存，所以在嵌入式环境里很常见，例如 uClibc 的 `qsort` 就是希尔排序。缺点是对近乎有序的数据不敏感：近乎有序时 62.86 ms，没比随机快多少。

3.4 交互实验

把数据切到“近乎有序”，对比插入排序和希尔排序的比较次数；再把规模拉到 48，看希尔的大步长怎样快速把“矮柱子”送到左边。

交互实验 • 排序过程可视化




```
for i = 1 to n-1:
  key = a[i]; j = i-1
  while j ≥ 0 and a[j] > key:
    a[j+1] = a[j]; j--       // 大的后移
  a[j+1] = key
```

点击「播放」或「单步」开始。

比较/读取　交换　写入　pivot　当前元素　已就位　不在当前子区间

面试注意

- “插入排序最好 O(n)”是对的，而且**不需要任何额外优化**，这一点和冒泡不同。
- 二分插入只减少比较次数，**移动次数仍是 O(n²)**，整体还是 O(n²)。
- 被问“希尔排序为什么比插入快”，要说到点上：一次远距离移动可以**消除多个逆序对**，而相邻移动一次只能消除一个。

本章速记

- 插入排序：代价 **O(n + 逆序对)**；稳定、原地、自适应、在线。
- 工业用途：快排的小区间收尾（阈值 16/32）、TimSort 的短段、帧间增量重排（E8：0.005 ms 对 0.062 ms）。
- 哨兵优化省掉了 `j ≥ 0` 判断，前提是比较器满足严格弱序。
- 希尔：分组插入，增量序列决定复杂度（Knuth / Hibbard 为 $O(n^{1.5})$）；不稳定。

PART THREE

第三部分　O(n log n) 三巨头：归并、快排、堆排

三者都达到了比较排序的下界，但各有取舍。归并用空间换来稳定和最坏保证，快排用最坏情况换来最快的平均速度，堆排用缓存友好性换来 O(1) 空间和最坏保证。

CHAPTER 04

归并排序：稳定、可靠、可以外排

分治的教科书范例：一分为二，各自排好，再把两个有序序列合并成一个。

本章目标

- 写出自顶向下和自底向上两种归并排序，理解合并时“相等取左”为什么保证稳定
- 会用递推式或递归树分析 O(n log n)，说清 O(n) 额外空间从哪来
- 掌握两个高频变形：链表排序（LeetCode 148）和逆序对计数

4.1 分而治之

归并排序分三步：**分**（从中间切开）、**治**（递归地排好两半）、**合**（把两个有序序列合并）。真正干活的是“合”。“分”只是算一下中点，一直切到每段只剩一个元素，单个元素天然有序。



图 4-1 归并排序的递归过程：一共 $\log_2 8 = 3$ 层，每层的合并总共处理 n 个元素

合并：双指针

两个有序序列 L 和 R 各放一个指针，每次比较两个指针指向的元素，把较小的那个输出，对应指针后移。一边走完以后，把另一边剩下的部分原样拷贝过去。

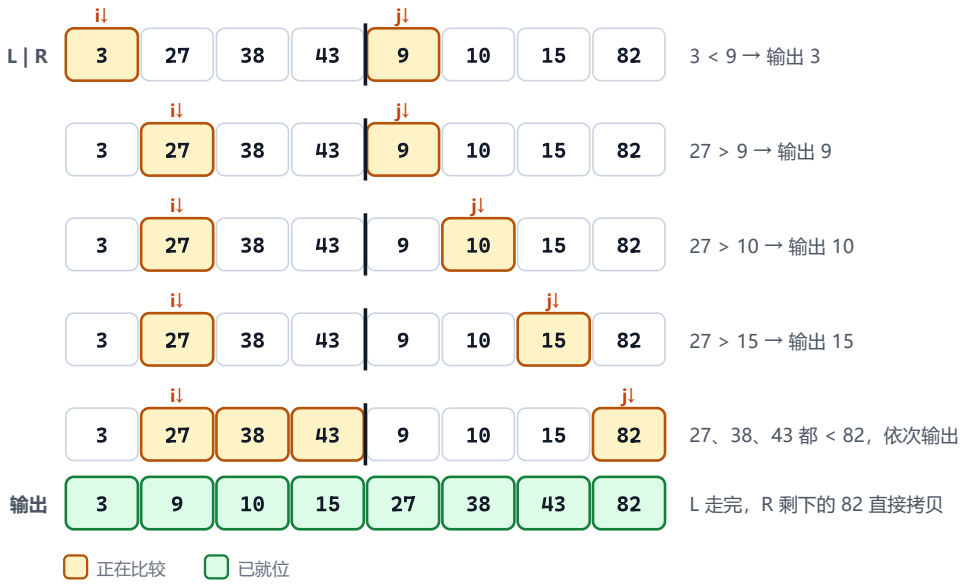


图 4-2　合并两个长度为 4 的有序序列：每次比较输出一个元素，最多比较 n−1 次

4.2 伪代码与 C++

归并排序 · 伪代码

```
mergeSort(a, lo, hi):                  // 区间 [lo, hi)
    if hi - lo < 2: return
    mid = lo + (hi - lo) / 2           // 不写 (lo+hi)/2，避免溢出
    mergeSort(a, lo, mid)
    mergeSort(a, mid, hi)
    merge(a, lo, mid, hi)

merge(a, lo, mid, hi):
    i = lo, j = mid, k = lo
    while i < mid and j < hi:
        if a[j] < a[i]: buf[k++] = a[j++]      // 只有右边“严格小于”才取右边
        else:           buf[k++] = a[i++]      // 相等取左边 → 稳定
    拷贝剩余部分：把 buf[lo, hi) 写回 a
```

merge_sort.h（自顶向下，缓冲区只分配一次）

```
template <class T, class Less>
void merge_rec(T* a, T* buf, int lo, int hi, Less& less) {   // [lo, hi)
    if (hi - lo ≤ 16) { insertion_sort(a + lo, hi - lo, less); return; }   // 优化 1：小区间插
    int mid = lo + (hi - lo) / 2;
    merge_rec(a, buf, lo, mid, less);
    merge_rec(a, buf, mid, hi, less);
    if (!less(a[mid], a[mid - 1])) return;                    // 优化 2：两半已经衔接有序，跳过合
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi)
        buf[k++] = less(a[j], a[i]) ? std::move(a[j++]) : std::move(a[i++]);
    while (i < mid) buf[k++] = std::move(a[i++]);
    while (j < hi)  buf[k++] = std::move(a[j++]);
    std::move(buf + lo, buf + hi, a + lo);
}

template <class T, class Less = std::less<T>>
void merge_sort(T* a, int n, Less less = {}) {
    std::vector<T> buf(n);                  // 只分配一次；千万别在每层递归里 new
    merge_rec(a, buf.data(), 0, n, less);
}
```

自底向上的版本不用递归：先把相邻的长度 1 的段两两合并，再合并长度 2 的段，然后是 4、8……配合**乒乓缓冲**（两块内存轮流做源和目标），还能省掉“写回”这一步。

merge_sort_bottom_up.h

```
template <class T, class Less = std::less<T>>
void merge_sort_bottom_up(T* a, int n, Less less = {}) {
    std::vector<T> buf(n);
    T* src = a;
    T* dst = buf.data();
    for (int width = 1; width < n; width *= 2) {
        for (int lo = 0; lo < n; lo += 2 * width) {
            int mid = std::min(lo + width, n), hi = std::min(lo + 2 * width, n);
            std::merge(src + lo, src + mid, src + mid, src + hi, dst + lo, less);   // std::me
        }
        std::swap(src, dst);                // 乒乓：这一轮的输出就是下一轮的输入
    }
    if (src ≠ a) std::copy(src, src + n, a);
}
```

4.3 复杂度与性质

递推式是 T(n) = 2T(n/2) + O(n)。用递归树看最直观：一共 $\log_2 n$ 层，每层的合并加起来正好处理 n 个元素，所以总共 O(n log n)。**它和输入无关**，最好、平均、最坏都是 n log n（如果加了“已衔接就跳过”的优化，有序输入会退化成 O(n)）。

性质	结论	说明
时间	O(n log n)，三种情况都是	比较次数在 ½ n $\log_2 n$ 到 n $\log_2 n$ 之间。E2（n=1000 随机）：9401 次，是所有算法里最少的
空间	O(n) 缓冲 + O(log n) 递归栈	数组版本最大的缺点
稳定	✓	前提是合并时“相等取左”。写成 `a[i] < a[j] ? 取左 : 取右` 就会在相等时取右，变成不稳定
自适应	加“已衔接跳过”后有一定自适应性	E2 有序输入：比较 999 次、写 0 次
访存	顺序读、顺序写	对缓存和预取都很友好，也适合磁盘、磁带

实测 E5：N = 100 万

本书实现的归并 70.07 ms（随机）/ 3.20 ms（有序），`std::stable_sort` 51.65 ms / 4.48 ms。MSVC 的 `stable_sort` 先用插入排序把每 32 个元素排好，再用乒乓缓冲合并，这是它比本书的教科书写法快约 26% 的主要原因，并且和 `std::sort`（51.38 ms）基本持平。

4.4 变形一：链表排序（LeetCode 148）

链表没法随机访问，快排找 pivot、堆排算 2i+1 都很别扭。**归并只需要顺序遍历和改指针**，而且合并时不需要额外缓冲，只是重新串链，所以链表排序的标准答案就是归并。`std::list::sort` 也是归并排序。

链表归并排序

```
struct Node { int val; Node* next; };

Node* merge(Node* a, Node* b) {
    Node dummy{0, nullptr};
    Node* t = &dummy;
    while (a && b) {
        if (b->val < a->val) { t->next = b; b = b->next; }
        else                 { t->next = a; a = a->next; }   // 相等取 a → 稳定
        t = t->next;
    }
    t->next = a ? a : b;
    return dummy.next;
}

Node* sort_list(Node* head) {
    if (!head || !head->next) return head;
    Node *slow = head, *fast = head->next;   // fast 从 head->next 出发：只有两个节点时也能正确断
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    Node* right = slow->next;
    slow->next = nullptr;                    // 断开成两条链
    return merge(sort_list(head), sort_list(right));
}
```

面试注意：快慢指针的起点

如果 `fast` 也从 `head` 出发，两个节点的链表中 `slow` 会停在第二个节点，`right` 为空，左半边仍是两个节点，于是无限递归。追问“空间复杂度”时，递归版是 O(log n) 栈；要做到 O(1)，得写自底向上的迭代版。

4.5 变形二：逆序对计数

合并时，如果右边的 `a[j]` 比左边的 `a[i]` 小，那么左边从 i 到 mid−1 的所有元素都比 `a[j]` 大，一次就能数出 `mid - i` 个逆序对。这样就能在 O(n log n) 内统计逆序对。

归并统计逆序对

```
long long count_inversions(int* a, int* buf, int lo, int hi) {   // [lo, hi)
    if (hi - lo < 2) return 0;
    int mid = lo + (hi - lo) / 2;
    long long c = count_inversions(a, buf, lo, mid) + count_inversions(a, buf, mid, hi);
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        if (a[j] < a[i]) { c += mid - i; buf[k++] = a[j++]; }   // a[i..mid) 全都比 a[j] 大
        else             { buf[k++] = a[i++]; }
    }
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf + lo, buf + hi, a + lo);
    return c;
}
```

游戏客户端视角：逆序对可以衡量“有多乱”

用逆序对数（或者抽样估计）判断一批数据离有序有多远，可以动态选择策略：几乎有序就做插入排序增量更新，很乱就整体 `std::sort`。渲染队列、排行榜的局部刷新都能用这个思路。

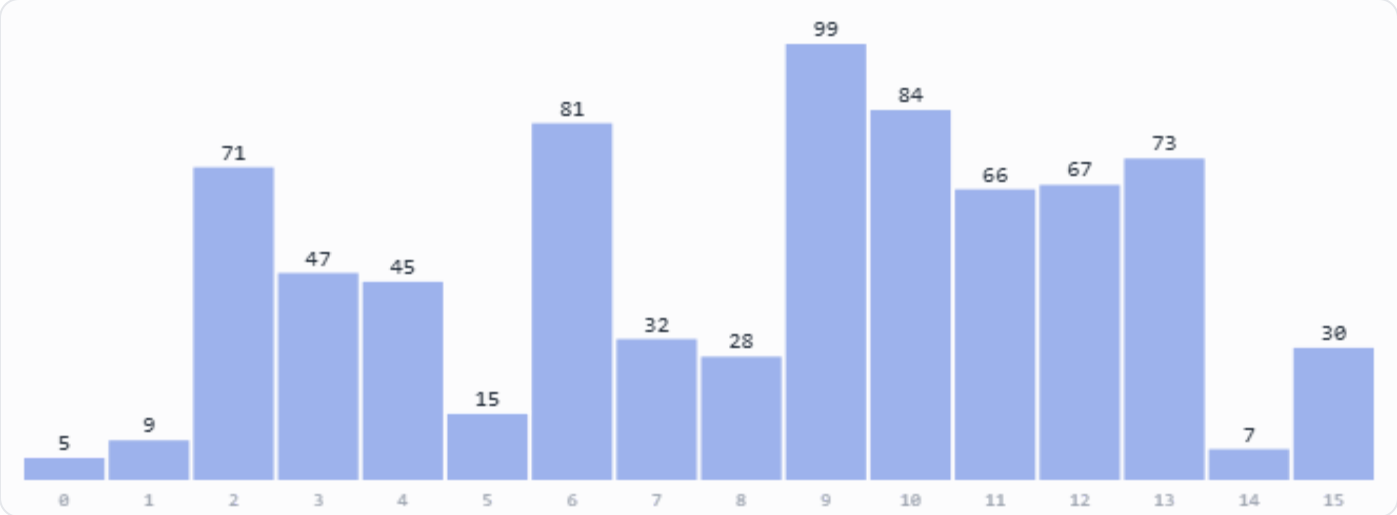
4.6 外部排序：数据大到内存放不下

归并的顺序访问特性，让它成为外部排序的基础：先把大文件切成能放进内存的块，每块排好后写回磁盘（称为“顺串”），再用一个**大小为 k 的小顶堆做 k 路归并**，每次从堆顶取出最小值输出。总复杂度是 O(n log n)，磁盘 IO 大约只需要 2 遍。游戏开发中，资源构建管线里对超大清单排序、对海量战斗日志做离线分析时会用到。

4.7 交互实验

注意观察浅蓝色背景表示的“当前子区间”：先一路缩小到长度 1，再逐层扩大合并。切到“大量重复”，看相等元素的先后顺序是否被保持。

交互实验 · 排序过程可视化




```
mergeSort(lo, hi):              // 区间 [lo, hi)
  if hi - lo < 2: return
  mid = (lo + hi) / 2
  mergeSort(lo, mid); mergeSort(mid, hi)
  tmp = a[lo..hi); i = lo; j = mid
  while i < mid and j < hi:
    a[k++] = (R[j] < L[i]) ? R[j++] : L[i++]   // 相等取左→稳定
  把剩下的一侧依次写回
```

点击「播放」或「单步」开始。

比较/读取 交换 写入 pivot 当前元素 已就位 不在当前子区间

面试注意

- 空间复杂度要答 **O(n)**，不是 O(n log n)。缓冲区只分配一次，每层复用。如果每次递归都 new 一个临时数组，那就是实现得不好。
- 原地归并存在（如 `std::inplace_merge` 在没有缓冲时的做法），但复杂度会变成 O(n log² n)，常数也大。`std::stable_sort` 申请不到内存时就会退回这种方案。
- 稳定性的关键就是一个符号：**相等取左**。手写时可以主动指出这一点。

本章速记

- 归并：**稳定**、三种情况都是 **O(n log n)**、额外 **O(n)** 空间、顺序访存。
- 工程优化：小区间插排、已衔接跳过合并、缓冲只分配一次、乒乓缓冲省去写回。

- 链表排序首选归并（`std::list::sort`）；合并时可顺便统计逆序对；外部排序用 k 路归并 + 小顶堆。

CHAPTER 05

快速排序：最快的平均，最危险的最坏

面试里被问得最多、追问得最深的排序。要讲清楚三件事：分区怎么写，pivot 怎么选，最坏情况怎么防。

本章目标

- 能手写 Lomuto 和 Hoare 两种分区，说出它们的差别和各自的坑
- 理解退化的原因：有序输入、大量重复、递归过深导致栈溢出（本章有实测）
- 掌握三路快排、递归短边、三数取中 / ninther 这些工程手段
- 能用快速选择在 O(n) 平均时间内求第 K 大

5.1 核心思想：分区

选一个基准值 pivot，把数组分成两部分：左边都 ≤ pivot，右边都 ≥ pivot，然后递归地处理两边。和归并相反，**快排的工作在“分”这一步做完，“合”什么都不用做。**分区完成时，左右两部分之间已经有序了。

它的效率完全取决于**分得是否均匀。**每次都对半分，递归深度是 $\log_2 n$，总共 O(n log n)；每次都分成 0 和 n−1，递归深度是 n，总共 $O(n^2)$。

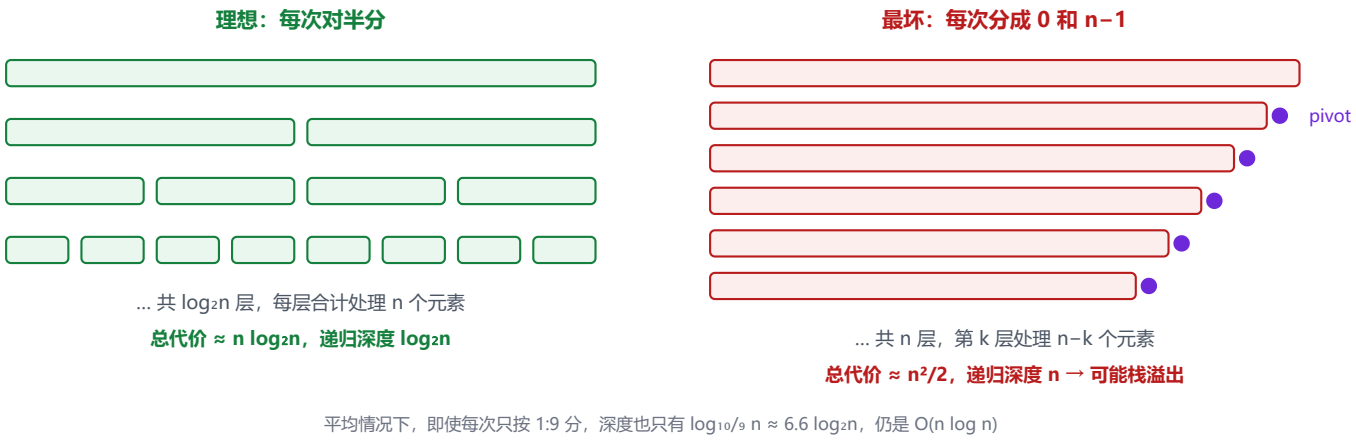


图 5-1　快排的递归树：分得均匀与否，决定了是 n log n 还是 n²

5.2 Lomuto 分区：单指针，好写但有坑

取最后一个元素作 pivot。用 i 标记“小于区”的下一个空位，j 从左往右扫描：遇到比 pivot 小的元素，就和 `a[i]` 交换，i 加 1。扫完以后，把 pivot 和 `a[i]` 交换，pivot 就到了它的最终位置。



图 5-2　Lomuto 分区（CLRS 的经典例子）：分区结束时 pivot 一定在它的最终位置上

Lomuto 分区

```
int partition_lomuto(int* a, int lo, int hi) {   // [lo, hi]，返回 pivot 的最终下标
    int p = a[hi], i = lo;
    for (int j = lo; j < hi; ++j)
        if (a[j] < p) std::swap(a[i++], a[j]);
    std::swap(a[i], a[hi]);
    return i;          // 递归 [lo, i-1] 和 [i+1, hi]，pivot 本身不再参与
}
```

Lomuto 有两个缺陷：一是交换次数多，每个比 pivot 小的元素都要换一次；二是**遇到大量相等元素会退化**。全部相等时 `a[j] < p` 永远为假，i 一直不动，每次都分成 0 和 n−1，于是变成 O(n²)。

5.3 Hoare 分区：双指针相向而行

Hoare 在 1961 年发明快排时用的就是这种分区：i 从左往右找 ≥ pivot 的元素，j 从右往左找 ≤ pivot 的元素，两边都找到就交换，直到 i、j 相遇。



图 5-3　Hoare 分区：递归 [lo, j] 和 [j+1, hi]，pivot 也参与递归

面试 5 分钟手写版（Hoare + 取中间值）

```
void quick_sort(int* a, int lo, int hi) {      // [lo, hi]
    if (lo ≥ hi) return;
    int p = a[lo + (hi - lo) / 2];             // 取中间位置的值：有序输入不会退化
    int i = lo - 1, j = hi + 1;
    while (i < j) {
        do ++i; while (a[i] < p);              // 严格小于：遇到等于 pivot 的也停下
        do --j; while (a[j] > p);
        if (i < j) std::swap(a[i], a[j]);
    }
    quick_sort(a, lo, j);                      // 用 j 划分
    quick_sort(a, j + 1, hi);
}
```

对比项	Lomuto	Hoare
指针	单向，i 慢 j 快	双向，相向而行
交换次数（随机）	约 n/2	约 n/6，少约 3 倍
全部相等	**退化 $O(n^2)$**	**每次都停下交换，正好对半分**
分区后 pivot	在最终位置，不再参与递归	不一定在最终位置，两侧都可能包含它
递归区间	[lo, i−1] 和 [i+1, hi]	[lo, j] 和 [j+1, hi]

面试注意：Hoare 的三个经典 bug

1. **用 `≤` 代替 `<`**：`while (a[i] ≤ p)` 在全等数组上会让 i 一路冲出右边界。
2. **pivot 取 `a[hi]` 又用 j 划分**：比如 `[1, 2]` 以 2 为 pivot，i 和 j 都停在下标 1，返回 j = 1，于是又递归 [0, 1]，形成死循环。用 j 划分时，pivot 必须取 `a[lo]` 或向下取整的中点。反过来，如果用 i 划分（[lo, i−1]、[i, hi]），pivot 就不能取 `a[lo]`。
3. **把 j 写成 i**：两种组合不能混用。最稳妥的做法是背熟上面这一套。

5.4 pivot 怎么选

策略	做法	弱点	谁在用
首 / 尾元素	直接取 `a[lo]` 或 `a[hi]`	有序、逆序输入直接 $O(n^2)$	教科书
中间元素	`a[(lo+hi)/2]`	可以被构造出最坏输入（如“风琴管”序列）	手写模板
随机	随机选一个下标	期望 O(n log n)；要调随机数，结果不可复现	算法课、面试
三数取中	lo、mid、hi 三者的中位数	仍可被构造攻击（McIlroy 1999），概率极低	libstdc++
九数取中（ninther）	取三组“三数取中”的中位数	更稳，代价是多做几次比较	MSVC（区间 > 40）、pdqsort（区间 > 128）

深入一层：确定性的 pivot 总能被攻击

McIlroy 在《A Killer Adversary for Quicksort》里构造了一个“对抗比较器”：它在排序过程中动态决定元素的大小，能让任何确定性选 pivot 的快排都跑出 $O(n^2)$。这在服务器上可能变成拒绝服务攻击。所以工业实现不靠 pivot 选得巧来保证最坏情况，而是**监控递归深度，超过阈值就换成堆排序**，这就是 introsort（第 8 章）。

5.5 退化实测：比较次数、递归深度、栈溢出

把朴素版（Lomuto + 末元素作 pivot）和工程版（三数取中 + Hoare + 小区间插排 + 递归短边）放在一起，分别喂随机和有序的数据：

E4：输入	朴素版比较次数	朴素版递归深度	工程版比较次数	工程版递归深度
N=1000 随机	10,453	18	11,171	4
N=1000 有序	**499,500**	**999**	7,188	6
N=16000 随机	268,630	35	258,659	8
N=16000 有序	**127,992,000**	**15,999**	179,068	10

有序的 16000 个数，朴素版比较了 1.28 亿次，正好是 n(n−1)/2；工程版只比较了 18 万次。在 E3 的计时中（N=20000），朴素版有序输入要 61.26 ms，工程版只要 0.06 ms，**相差约 1000 倍。**

E4 · 把朴素快排放进不同大小的线程栈

```
快排-朴素 有序 N=  2000，线程栈  64KB：【栈溢出 STACK_OVERFLOW】，最大递归深度 502
快排-朴素 有序 N= 20000，线程栈  64KB：【栈溢出 STACK_OVERFLOW】，最大递归深度 495
快排-朴素 有序 N= 10000，线程栈 1024KB：正常完成，最大递归深度 9999
快排-朴素 有序 N= 20000，线程栈 1024KB：【栈溢出 STACK_OVERFLOW】，最大递归深度 10732
快排-工程 有序 N=1000000，线程栈  64KB：正常完成，最大递归深度 16
```

每层递归大约占 100 字节栈空间。Windows 主线程默认栈是 1 MB，**只要 2 万个有序元素就能把它撑爆。**游戏引擎的 Job 线程、协程或纤程的栈通常更小（几十 KB 很常见），几百层就溢出。递归短边的工程版排 100 万个元素，最大深度只有 16。

游戏客户端视角：线上崩溃的一种典型来源

“策划配了一张已经排好序的表，加载时用自己写的快排再排一次，在 Job 线程里栈溢出。”这类问题在开发机上很难复现，因为开发机的数据往往是乱序的，或者跑在主线程上栈更大。**结论：不要在引擎代码里手写快排，用 `std::sort`。真要手写，就必须递归短边并加深度限制。**

5.6 递归短边、循环长边：把栈深压到 O(log n)

每次分区后，只对**较短的一边**递归，较长的一边改成在当前函数里用循环继续处理。每次递归调用处理的区间长度都不超过父区间的一半，所以递归深度 ≤ $\log_2 n$，和输入无关。注意，这只保证了栈深，时间复杂度最坏仍然是 $O(n^2)$。

工程版快排（三数取中 + Hoare + 小区间插排 + 递归短边）

```cpp
template <class T, class Less = std::less<T>>
void quick_sort_engineered(T* a, int lo, int hi, Less less = {}) {   // [lo, hi]
    while (hi - lo > 16) {
        int mid = lo + (hi - lo) / 2;
        // 三数取中：结束后 a[lo] ≤ a[mid] ≤ a[hi]，两端还能起到哨兵作用
        if (less(a[mid], a[lo])) std::swap(a[mid], a[lo]);
        if (less(a[hi],  a[lo])) std::swap(a[hi],  a[lo]);
        if (less(a[hi],  a[mid])) std::swap(a[hi], a[mid]);
        T p = a[mid];
        int i = lo - 1, j = hi + 1;
        for (;;) {
            do ++i; while (less(a[i], p));
            do --j; while (less(p, a[j]));
            if (i ≥ j) break;
            std::swap(a[i], a[j]);
        }
        if (j - lo < hi - j) { quick_sort_engineered(a, lo, j, less); lo = j + 1; }   // 递归短
        else                 { quick_sort_engineered(a, j + 1, hi, less); hi = j; }   // 长边
    }
    insertion_sort(a + lo, hi - lo + 1, less);   // ≤ 17 个元素：插入排序收尾
}
```

5.7 三路快排：专治大量重复

游戏数据里重复键非常多：品质只有 5 档、职业只有 6 种、队伍只有 2 个。Dijkstra 的“荷兰国旗”分区把数组分成三段：**< p、= p、> p**。中间那段已经就位，不再参与递归。重复越多，递归规模缩小得越快。

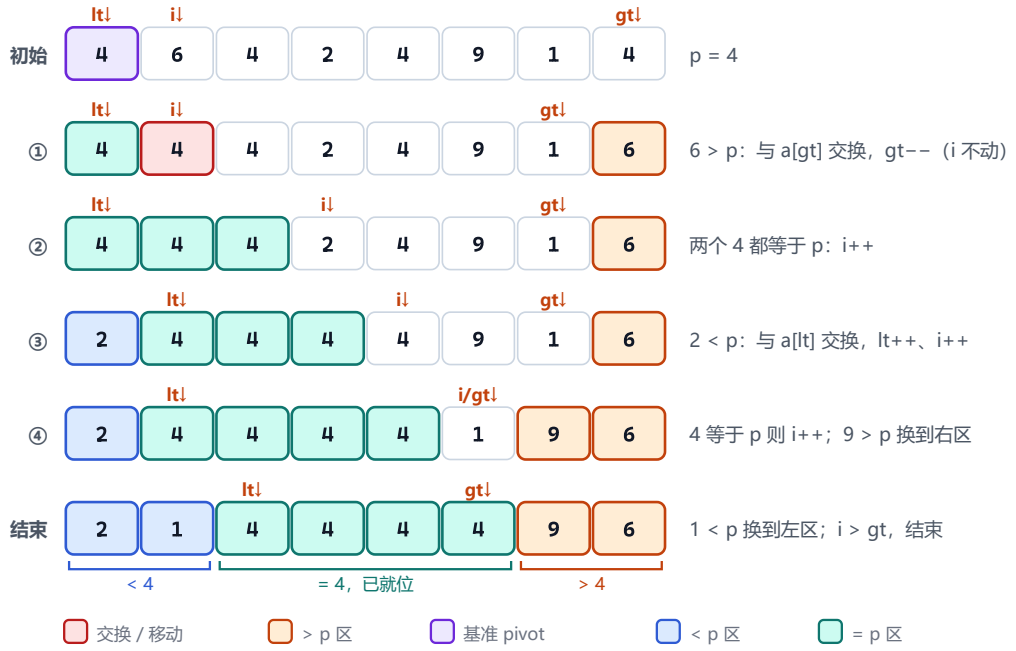


图 5-4　三路划分：不变式 [lo, lt) < p，[lt, i) = p，(gt, hi] > p，[i, gt] 尚未处理

三路快排

```cpp
template <class T, class Less = std::less<T>>
void quick_sort_3way(T* a, int lo, int hi, Less less = {}) {    // [lo, hi]
    if (lo ≥ hi) return;
    std::swap(a[lo], a[lo + rand() % (hi - lo + 1)]);           // 随机 pivot（示意；工程上用 <rc
    T p = a[lo];
    int lt = lo, i = lo + 1, gt = hi;
    while (i ≤ gt) {
        if (less(a[i], p))      std::swap(a[lt++], a[i++]);
        else if (less(p, a[i])) std::swap(a[i], a[gt--]);       // 换过来的元素还没看过，所以 i 不
        else                    ++i;
    }
    quick_sort_3way(a, lo, lt - 1, less);                       // [lt, gt] 全等于 p，跳过
    quick_sort_3way(a, gt + 1, hi, less);
}
```

实测 E5：N = 100 万，三路的长处和代价

大量重复（16 种值）：三路 7.95 ms，工程版 Hoare 16.29 ms，`std::sort` 9.68 ms。**有序**：三路 20.84 ms，Hoare 3.94 ms。三路每处理一个元素都要做两次比较、交换也更多，所以没有重复时反而更慢。MSVC 的 `std::sort` 在分区时会把等于 pivot 的元素聚到中间（`_Partition_by_median_guess_unchecked` 返回一个相等区间），兼顾了两种情况。

5.8 为什么快排在实践中最快

快排的平均比较次数约为 $1.39\, n \log_2 n$，比归并的 $\sim n \log_2 n$ 还多。它快在每一步的代价小：

- **内层循环极简**：和放在寄存器里的 pivot 比较，然后指针加 1。
- **访存顺序**：两个指针各自顺序扫描，硬件预取效果好；区间缩小到能放进 L1 以后，后面的操作全部命中缓存。
- **写得少**：E2（n=1000 随机）中，工程版快排写 6730 次，归并写 18686 次，堆写 11585 次。
- **原地**：不需要额外缓冲，没有分配开销。

5.9 快速选择：O(n) 求第 K 大

只想要第 K 小的元素时，分区以后只需要进入**包含 K 的那一边**，另一边直接丢掉。平均工作量是 $n + n/2 + n/4 + \ldots \approx 2n$，即 $O(n)$。最坏仍是 $O(n^2)$；用中位数的中位数（BFPRT）选 pivot 可以保证最坏 $O(n)$，但常数太大，工程上很少用。`std::nth_element` 是 introselect：快速选择 + 深度兜底。

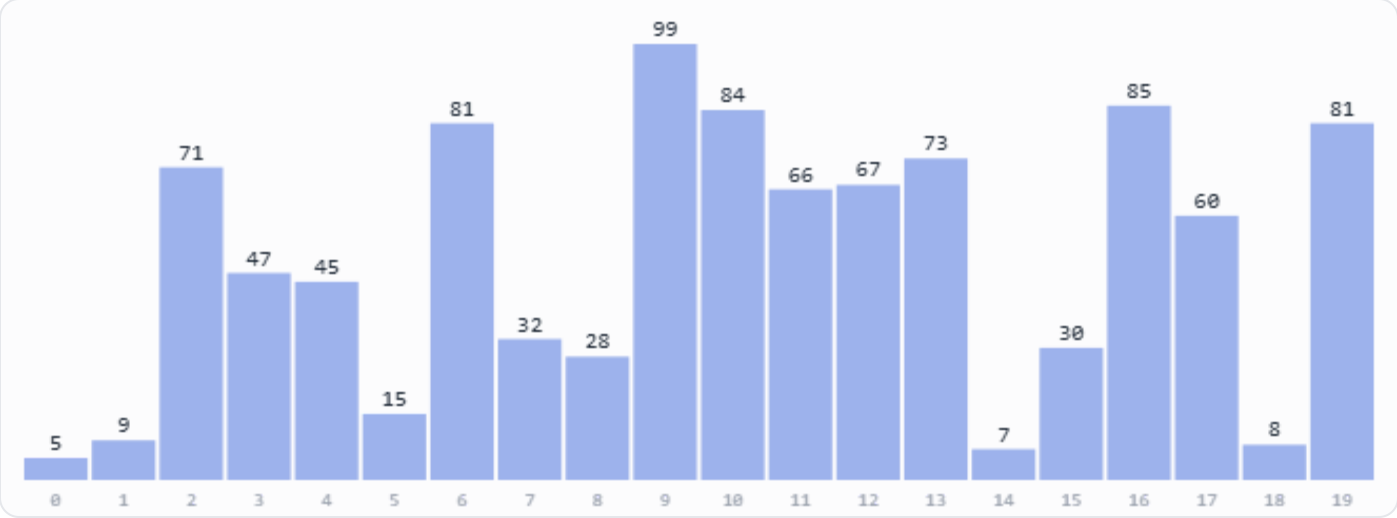
快速选择（LeetCode 215 的核心）

```cpp
int quick_select(std::vector<int>& a, int k) {   // 返回第 k 小（k 从 0 开始），会打乱 a
    int lo = 0, hi = int(a.size()) - 1;
    while (lo < hi) {
        int p = a[lo + (hi - lo) / 2], i = lo - 1, j = hi + 1;
        while (i < j) {
            do ++i; while (a[i] < p);
            do --j; while (a[j] > p);
            if (i < j) std::swap(a[i], a[j]);
        }
        if (k ≤ j) hi = j; else lo = j + 1;   // 只保留包含 k 的那一边
    }
    return a[k];
}
```

5.10 交互实验：三种快排对比

先选“快排 · Lomuto”，数据切到“已有序”，观察每次分区后右侧都是空的、递归一层只少一个元素；再换成“Hoare + 三数取中”。最后选“大量重复”，对比三路划分。

交互实验 · 排序过程可视化




```
quick(lo, hi):
  if lo ≥ hi: return
  p = a[hi]; i = lo          // 末元素作 pivot
  for j = lo to hi-1:
    if a[j] < p:
      swap(a[i], a[j]); i++
  swap(a[i], a[hi])          // pivot 归位
```

```
quick(lo, i-1); quick(i+1, hi)
```

点击「播放」或「单步」开始。

比较/读取 交换 写入 pivot 当前元素 已就位 不在当前子区间

面试注意：快排高频追问清单

- **最坏情况什么时候出现？** pivot 总是取到最大或最小值：首 / 尾作 pivot 遇到有序或逆序输入；Lomuto 遇到全相等的输入。
- **空间复杂度？** 平均 O(log n)，朴素实现最坏 O(n)；递归短边可以保证 O(log n)。**不是 O(1)。**
- **稳定吗？** 不稳定，因为分区时有远距离交换。
- **和堆排比？** 两者平均都是 n log n，但快排缓存友好、写得少，实测更快（E5：47 ms 对 74 ms）；堆排的优势是最坏有保证、O(1) 空间。
- **怎么避免最坏情况？** 随机 / 三数取中 / ninther 选 pivot 只能降低概率；真正的保证要靠 introsort 的深度限制，切换到堆排。

本章速记

- Lomuto：好写，pivot 最终归位；交换多，全相等时退化。Hoare：交换少约 3 倍，全相等时对半分；pivot 不一定归位，递归 [lo, j]、[j+1, hi]。
- 退化实测：有序的 16000 个数，朴素版比较 1.28 亿次、递归 15999 层；1 MB 栈遇到 2 万个有序元素就溢出。
- 工程四件套：**三数取中 / ninther、Hoare、小区间插排、递归短边**；再加深度限制 + 堆排兜底，就是 introsort。
- 三路快排专治重复键（E5：7.95 ms 对 16.29 ms）；快速选择平均 O(n)。

CHAPTER 06

堆排序：最坏有保证的原地排序

把数组看成一棵完全二叉树，反复取出堆顶的最大值放到末尾。理论上很完美，实际跑起来却不快。

本章目标

- 掌握数组和完全二叉树之间的下标映射，能写出 siftDown
- 理解为什么自底向上建堆是 O(n)，而不是 O(n log n)
- 说清堆排为什么比快排慢，以及它在哪些地方不可替代

6.1 二叉堆：用数组存一棵完全二叉树

大顶堆：每个节点都不小于它的孩子，所以堆顶 `a[0]` 是最大值。因为是完全二叉树，节点可以按层序紧密地存在数组里，不需要指针：

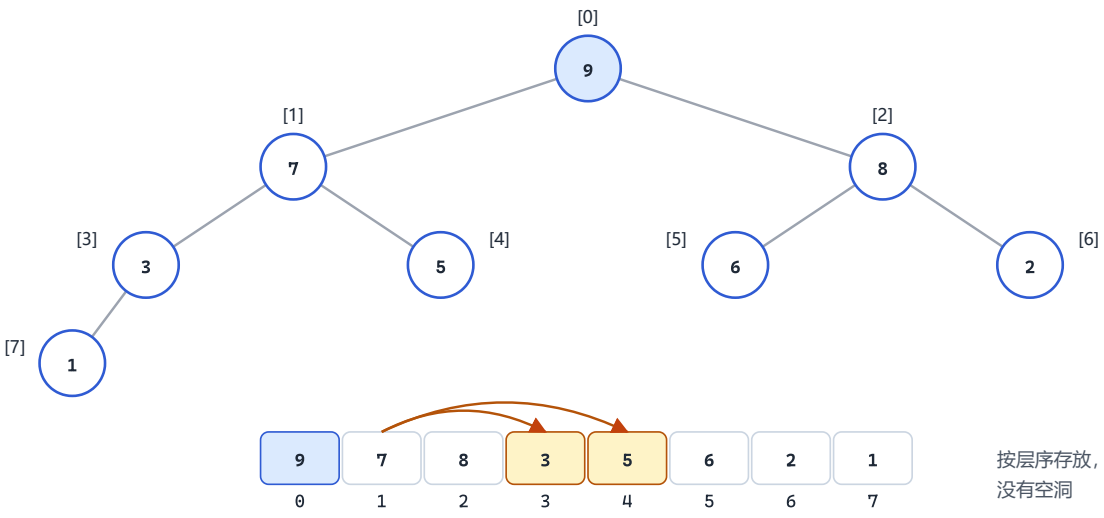


图 6-1　大顶堆 [9,7,8,3,5,6,2,1]：树和数组是同一份数据的两种看法

6.2 两个阶段：建堆，然后反复取堆顶

1. **建堆**：从最后一个非叶子节点 `n/2-1` 开始往前，对每个节点做 siftDown（下沉）。全部做完，整个数组就是一个大顶堆。
2. **排序**：把堆顶（最大值）和堆的最后一个元素交换，最大值就到了最终位置；堆的大小减 1，对新的堆顶做 siftDown 恢复堆的性质。重复 n−1 次。



图 6-2　堆排序的前两轮：每轮把最大值换到末尾，再让新的堆顶下沉，一共下沉 O(log n) 层

heap_sort.h

```cpp
template <class T, class Less>
void sift_down(T* a, int i, int n, Less& less) {   // 在 a[0, n) 中让 a[i] 下沉
    T v = std::move(a[i]);
    for (;;) {
        int c = 2 * i + 1;
        if (c ≥ n) break;
        if (c + 1 < n && less(a[c], a[c + 1])) ++c;    // 取较大的孩子
        if (!less(v, a[c])) break;                      // v ≥ 孩子：找到位置
        a[i] = std::move(a[c]);                         // 孩子上移（“挖坑”法，比反复 swap 少一半
        i = c;
    }
    a[i] = std::move(v);
}

template <class T, class Less = std::less<T>>
void heap_sort(T* a, int n, Less less = {}) {
    for (int i = n / 2 - 1; i ≥ 0; --i) sift_down(a, i, n, less);   // 建堆：O(n)
    for (int end = n - 1; end > 0; --end) {
        std::swap(a[0], a[end]);                                    // 最大值归位
        sift_down(a, 0, end, less);                                 // O(log n)
    }
}
// 等价的 STL 写法：std::make_heap(b, e); std::sort_heap(b, e);
```

6.3 为什么建堆是 O(n)

直觉上，n/2 个节点各下沉 O(log n)，应该是 O(n log n)。但大多数节点都在底层，下沉不了几层：高度为 h 的节点最多有 $n/2^{h+1}$ 个，每个最多下沉 h 层，所以总代价是

$$\Sigma_{h\geq 0} \ (n \ / \ 2^{h+1}) \ \cdot \ h \ = \ n \ \cdot \ \Sigma \ h/2^{h+1} \ = \ n \ \cdot \ 1 \ = \ O(n)$$

一半的节点是叶子，根本不用动；四分之一的节点只下沉 1 层……真正下沉 log n 层的只有根节点一个。**反过来，如果一个个 push 进去建堆（上浮），就是 O(n log n)**，因为多数节点在底部，要上浮很多层。这是一个常见的追问点。

6.4 复杂度与性质

性质	结论
时间	最好、平均、最坏都是 O(n log n)；比较次数约 2n $\log_2 n$（E2：n=1000 随机，16840 次，约为归并的 1.8 倍）
空间	**O(1)**，迭代实现，没有递归，这是它最大的优点
稳定	✗：堆顶和堆尾的交换跨度非常大（E1 逆序对 2916，所有算法里最多）
自适应	✗：输入有序也一样要建堆、拆堆（E5：有序 35.90 ms，快排 3.94 ms）

6.5 为什么堆排在实践中慢



图 6-3 同为 O(n log n)，堆排比快排慢了约 56%

- **缓存不友好**：下沉时从 i 跳到 2i+1，越往下跨度越大。100 万个 int 是 4 MB，下层节点之间相隔几十 KB 甚至几 MB，几乎每次访问都会缓存未命中。快排的两个指针都是顺序扫描。
- **比较次数多**：每下沉一层要比较两次（先比较两个孩子，再和孩子比较），总数约 2n log n。
- **分支难预测**：“左孩子大还是右孩子大”基本是随机的。
- **拆堆阶段反复把小元素换到堆顶**，它几乎一定会一路沉到底，做了很多无用的比较。

深入一层：两种改进

Floyd 的自底向上堆排（bottom-up heapsort）：既然换上来的元素多半会沉到底，那就先沿着“较大孩子”的路径直接走到叶子（每层只比较一次），再从叶子往上找它真正的位置。比较次数能降到约 n $\log_2 n$。**d 叉堆**（如 4 叉堆）：树高变成 $\log_4 n$，一个节点的孩子在内存里相邻，缓存表现更好，优先队列常用这种结构。

6.6 不可替代的场合

introsort 的兜底

快排递归深度超过 $2\lfloor \log_2 n \rfloor$ 时，`std::sort` 对当前区间改用堆排，从而保证最坏 O(n log n)，而且不需要额外内存。

内核和嵌入式

Linux 内核的 `lib/sort.c` 是堆排：没有递归，不会撑爆很小的内核栈；O(1) 内存；最坏情况可预测。

Top-K / partial_sort

维护大小为 K 的堆，扫描一遍是 O(n log K)。E9：100 万个积分取前 100，小顶堆 0.41 ms，全量排序 58.61 ms。

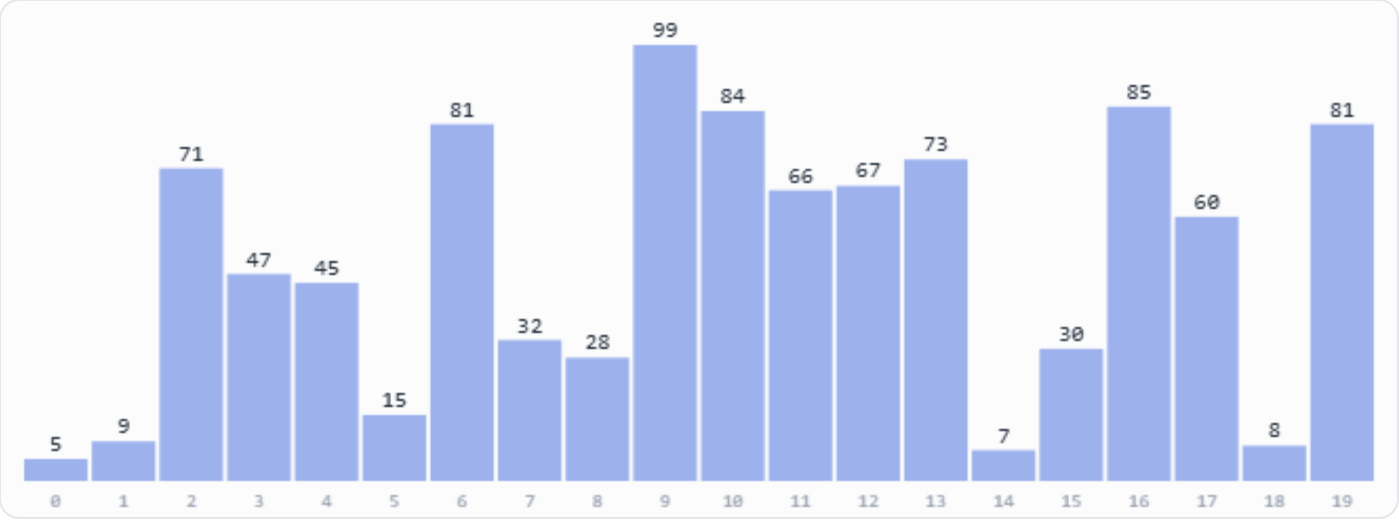
游戏里的优先队列

技能 / Buff 定时器（按到期时间的小顶堆）、A* 的 open list、事件调度、k 路归并，本质上都是堆。

6.7 交互实验

注意看两个阶段：前半段在建堆，柱子没有明显规律；后半段每轮都把当前最高的柱子换到右侧，变绿就位。

交互实验 · 排序过程可视化




```
for i = n/2-1 downto 0: siftDown(i, n)    // 建大顶堆 O(n)
for end = n-1 downto 1:
  swap(a[0], a[end])         // 堆顶(最大)放到末尾
  siftDown(0, end)
siftDown(i, n):
  while 2i+1 < n:
    c = 2i+1 与 2i+2 中较大者
```

```
    if a[i] ≥ a[c]: break
    swap(a[i], a[c]); i = c
```

点击「播放」或「单步」开始。

比较/读取 交换 写入 pivot 当前元素 已就位 不在当前子区间

面试注意

- 建堆 O(n) 要会证明，至少要能说出“大多数节点在底层，下沉不了几层”。
- 升序排序用**大顶堆**（每轮把最大值放到末尾）。很多人会下意识地说“小顶堆”，那样得到的是降序。
- `std::priority_queue` 默认是大顶堆；要小顶堆得写 `std::priority_queue<int, std::vector<int>, std::greater<int>>`。

本章速记

- 数组下标：父 (i−1)/2，孩子 2i+1、2i+2；升序用大顶堆。
- 自底向上建堆 **O(n)**；排序阶段 n−1 次下沉，每次 O(log n)。
- 最坏 O(n log n)、**O(1) 空间**、不稳定、不自适应；缓存不友好，实测比快排慢约 56%。
- 用途：introsort 兜底、内核排序、Top-K、优先队列、定时器。

PART FOUR

第四部分　非比较排序：绕开 n log n

不再问“谁大谁小”，而是直接利用键的数值结构：用键做数组下标，或者按数位逐位分配。代价是对键有要求，并且要额外的内存。

CHAPTER 07

计数排序、基数排序、桶排序

三者都是“分配式”排序。计数排序是基础，基数排序是多轮计数排序，桶排序是按值域粗分以后再细排。

本章目标

- 写出稳定的计数排序，理解前缀和与“倒序回填”各自的作用
- 理解 LSD 基数排序为什么要求每一趟都稳定，并能对负数、浮点数排序
- 知道桶排序在什么分布下有效、什么情况下会退化
- 明白它们在游戏里的典型用途：渲染 sort key、分组、直方图

7.1 计数排序：直接把值当下标

如果键是 [0, k) 范围内的整数，可以开一个大小为 k 的计数数组，数一下每个值出现了几次，再按值从小到大输出。只要统计和输出两遍扫描，时间 O(n + k)。

只输出“值”时直接按计数展开就行。但如果要排的是带着其他字段的**记录**，并且要求稳定，就需要“前缀和 + 倒序回填”。下例中 a/b/c 用来区分键相同的记录：

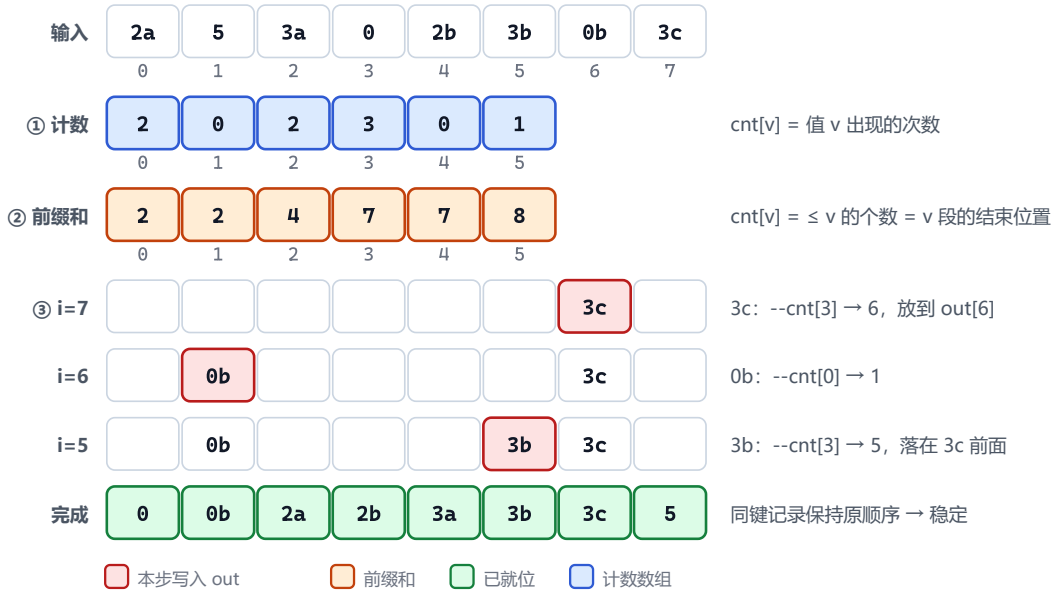


图 7-1　稳定的计数排序：倒序扫描输入，每个值从它那一段的末尾往前填

counting_sort.h（稳定版）

```cpp
// key(x) 返回 [0, k) 的整数
template <class T, class Key>
void counting_sort(std::vector<T>& a, int k, Key key) {
    std::vector<int> cnt(k + 1, 0);
    for (const T& x : a) ++cnt[key(x) + 1];             // 存到 cnt[v+1]
    for (int v = 0; v < k; ++v) cnt[v + 1] += cnt[v];   // 前缀和：cnt[v] 变成 v 段的起始位置
    std::vector<T> out(a.size());
    for (const T& x : a) out[cnt[key(x)]++] = x;        // 正序扫描 + 从起始位置往后填，同样稳定
    a.swap(out);
}
```

图 7-1 用的是“结束位置 + 倒序扫描”（教材常见写法），代码用的是“起始位置 + 正序扫描”，两者等价，都是稳定的。**不能混用**：结束位置配正序扫描，同键记录会被反过来。

性质	结论
时间	O(n + k)，和输入顺序无关
空间	O(n + k)：输出数组 + 计数数组
稳定	✓（按上面的写法）
适用	k 和 n 同一量级或更小。k 远大于 n 时（比如对 100 个 32 位整数排序）就得不偿失

游戏客户端视角：计数排序到处都是，只是不叫这个名字

- 背包按品质（白 / 绿 / 蓝 / 紫 / 橙，k=5）分组；英雄列表按职业分组。
- 粒子按发射器分组、实例按网格 ID 分批，都是“先计数，再前缀和得到偏移，再回填”。GPU 上用 compute shader 做的直方图 + prefix sum 完全是同一套思路。
- 空间哈希网格：每个格子统计一下物体数，做前缀和，就得到一个紧凑的“格子 → 物体列表”数组，比给每个格子开一个 `std::vector` 对缓存友好得多。

7.2 基数排序：按位多轮计数

键的值域太大（比如 32 位整数）时，直接计数需要 40 亿个计数器。基数排序把键拆成若干“位”（比如每 8 比特一位，共 4 位），**从最低位开始，每一位做一次稳定的计数排序**，这叫 LSD（Least Significant Digit first）基数排序。



图 7-2 LSD 基数排序（十进制演示）：每一趟都必须稳定，低位排好的顺序才会被保留下来

为什么每一趟都必须稳定？以 170 和 75 为例：按十位排时，两者十位都是 7，平局。它们的个位顺序（0 < 5）是上一趟排好的，只有稳定排序才能把这个顺序保留下来。如果这一趟不稳定，75 就可能跑到 170 前面，结果错误。这正是第 1 章说的多关键字排序：**先按次要键排，再用稳定排序按主要键排。**

radix_sort.h（8 比特一趟，可跳过无用趟）

```
// 对 uint64_t 键做 LSD 基数排序；tmp 是调用方提供的、和 a 等长的缓冲（可每帧复用）
template <class T, class KeyFn>
void radix_sort_u64(T* a, T* tmp, size_t n, KeyFn key) {
    T* src = a; T* dst = tmp;
    for (int pass = 0; pass < 8; ++pass) {
        const int shift = pass * 8;
        uint32_t cnt[257] = {};
        for (size_t i = 0; i < n; ++i) ++cnt[((key(src[i]) >> shift) & 0xFF) + 1];
        if (std::find(cnt + 1, cnt + 257, uint32_t(n)) == cnt + 257) continue;   // 这一字节全
        for (int b = 0; b < 256; ++b) cnt[b + 1] += cnt[b];
        for (size_t i = 0; i < n; ++i) dst[cnt[(key(src[i]) >> shift) & 0xFF]++] = src[i];
        std::swap(src, dst);                                                     // 乒乓缓冲
    }
    if (src != a) std::memcpy(a, src, n * sizeof(T));   // 示意；非平凡类型请用 std::move
}
```

复杂度：O(d • (n + b))

d 是趟数，b 是每一位的取值个数（基数）。32 位整数用 8 比特一位，d = 4、b = 256，复杂度就是 O(4 · (n + 256))，对大 n 而言是线性的。基数选得越大，趟数越少，但计数数组越大：256 个计数器正好能放进 L1；如果选 16 比特一位（65536 个计数器），计数数组会超出 L1，反而变慢。

实测 E5：N = 100 万个随机 int

基数排序 **4.31 ms**，`std::sort` 51.38 ms，快了约 12 倍。但它**不是自适应的**：输入有序时基数排序要 5.97 ms，`std::sort` 只要 3.91 ms。小数据量时优势也不明显，E8 中 N=2000 时二者分别是 0.016 ms 和 0.023 ms，基本持平。

负数和浮点数

LSD 基数排序是按无符号整数的二进制位排序的，所以要先把键**映射成一个保持大小顺序的无符号整数**：

把有符号整数、浮点数映射成可排序的无符号键

```
inline uint32_t key_of(int32_t x) {
    return uint32_t(x) ^ 0x80000000u;          // 翻转符号位：负数变到 [0, 2^31)，正数变到 [2^31,
}

inline uint32_t key_of(float f) {
    uint32_t u;
    std::memcpy(&u, &f, 4);                    // 别用 *(uint32_t*)&f：违反严格别名规则
    return (u & 0x80000000u) ? ~u              // 负数：全部取反（负得越多，原始位模式越大）
                             : u | 0x80000000u; // 正数：只翻转符号位
}
// 渲染里的深度一般非负，可以直接把 float 位模式当键；NaN 要提前处理掉
```

LSD 和 MSD

	LSD（从低位开始）	MSD（从高位开始）
做法	每一趟处理整个数组	按最高位分桶，再在每个桶内递归处理下一位
要求	每一趟必须稳定	桶内递归，不要求稳定
适合	定长键：整数、64 位 sort key	变长键：字符串；可以提前结束（桶里只剩 1 个元素）
实现	简单，无递归，适合 GPU	递归；小桶要转插入排序

7.3 桶排序：按值域均匀分桶

假设数据在 [0, 1) 内**均匀分布**。开 n 个桶，第 i 个桶装 [i/n, (i+1)/n) 范围的数，平均每个桶只有 1 个元素；对每个桶做插入排序，再按桶的顺序拼起来。



图 7-3 桶排序（CLRS 的经典例子）：分布均匀时每个桶只有很少的元素，桶内排序几乎不花时间

- **期望时间 O(n)**：要求数据均匀分布。**最坏 O(n²)**：所有数据挤进同一个桶，退化成一次插入排序。
- 是否稳定取决于桶内用的算法：桶内用插入排序就是稳定的。
- 工程上不要给每个桶开一个 `std::vector`，每个桶都要分配内存，非常慢。本书 demo 先计数、再前缀和、再回填到一个扁平数组，最后在每段内做插入排序。实测 E5：100 万个均匀分布的 float，桶排序 9.68 ms，`std::sort` 63.61 ms。

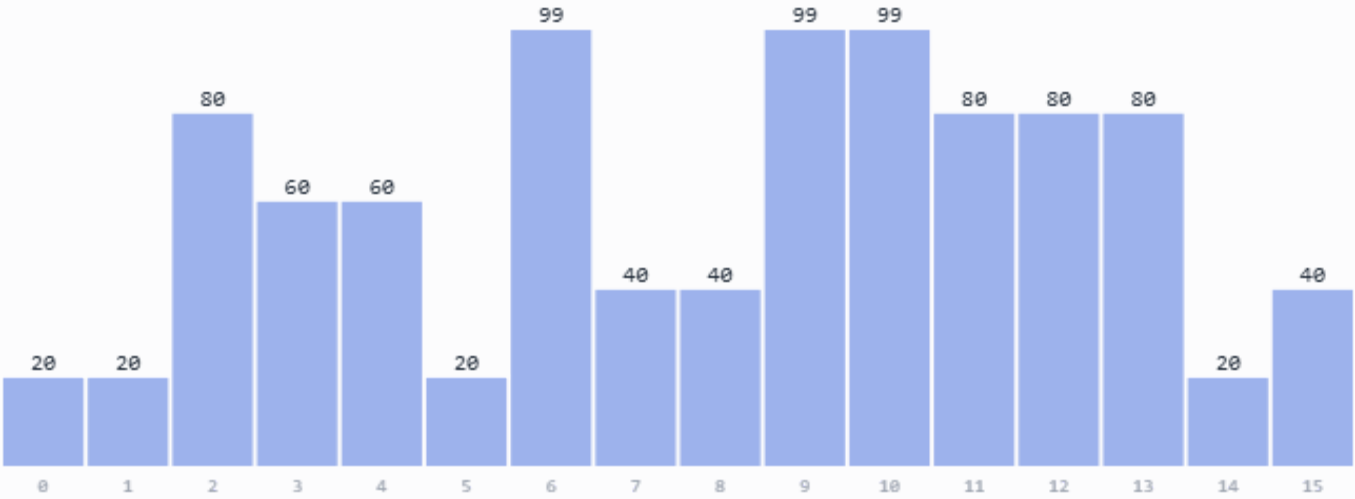
游戏客户端视角：透明物体的“深度分桶”

透明物体需要从远到近绘制。如果数量很多（比如成千上万个粒子），可以把视锥的深度范围分成 k 个桶，只按桶的顺序绘制，桶内不再排序。这是一个用少量误差换取速度的近似排序。GPU 粒子系统常在 compute shader 里做双调排序（bitonic sort）或基数排序，原因是它们的数据访问模式是固定的，适合并行。

7.4 交互实验

计数排序那一栏显示的是输出数组的填充过程：注意它是从右往左（倒序扫描输入）填的。基数排序用的是 3 位十进制数，观察每趟之后“低几位”逐渐有序。

交互实验 • 排序过程可视化




```
cnt[0..k] = 0
for x in a: cnt[x]++
for v = 1..k: cnt[v] += cnt[v-1]    // 前缀和 = 该值的结束位置
for i = n-1 downto 0:               // 倒序扫描 → 稳定
  out[--cnt[a[i]]] = a[i]
a = out
```

点击「播放」或「单步」开始。

比较/读取 交换 写入 pivot 当前元素 已就位 不在当前子区间

面试注意

- “基数排序 O(n)，所以总比快排快”——**不对。**它有前提：键是定长整数或可以映射成定长整数，数据量足够大，有 O(n) 的额外内存。数据量小或基本有序时它可能更慢（E5 有序：5.97 ms 对 3.91 ms）。
- 被问“基数排序为什么从低位开始”：因为从低位开始时，每一趟都处理整个数组，只要求每趟稳定，不需要递归；从高位开始就要在每个桶内递归。
- 被问“1 亿个 0~100 岁的年龄怎么排”：计数排序，k = 101，一遍计数一遍展开，O(n)，计数数组只有 101 个 int。

本章速记

- 计数排序：O(n + k)，稳定的关键是**前缀和 + 回填方向配对**（起始位置 + 正序，或结束位置 + 倒序）。
- LSD 基数排序：O(d(n + b))，**每一趟必须稳定**；8 比特一趟，256 个计数器正好放进 L1。
- 负整数翻转符号位，浮点数“负数全取反、正数翻符号位”，就能按无符号整数排序。

- 桶排序：均匀分布时期望 $O(n)$，最坏 $O(n^2)$；用扁平数组实现。

PART FIVE

第五部分　工业实现：标准库里的排序

面试中“说说 std::sort 的实现”几乎是必考题。知道它是 introsort 只是第一步，还要清楚阈值、兜底、比较器的约束，以及各种 STL 排序接口应该怎么选。

CHAPTER 08

std::sort 家族与严格弱序

std::sort 是快排、堆排、插排三者的组合；stable_sort 是带缓冲的归并；partial_sort 是堆；nth_element 是快速选择。

本章目标

- 讲清 introsort 的三段结构，以及 MSVC 和 libstdc++ 的具体参数
- 会根据需求在 sort / stable_sort / partial_sort / nth_element 之间选择
- 理解严格弱序的四条要求，知道违反它在实测中会发生什么
- 了解 pdqsort、TimSort 这些现代排序算法改进了什么

8.1 introsort：快排做主体，堆排兜底，插排收尾

introsort（内省排序）由 David Musser 在 1997 年提出。它解决的问题是：快排平均最快，但最坏 $O(n^2)$。办法是**一边快排一边数递归深度**，深度超过阈值，说明 pivot 一直选得不好，就对当前子区间改用堆排序。C++11 起，标准要求 `std::sort` 的复杂度为 O(n log n)（以前只要求平均），这实际上就要求实现采用 introsort 一类的做法。

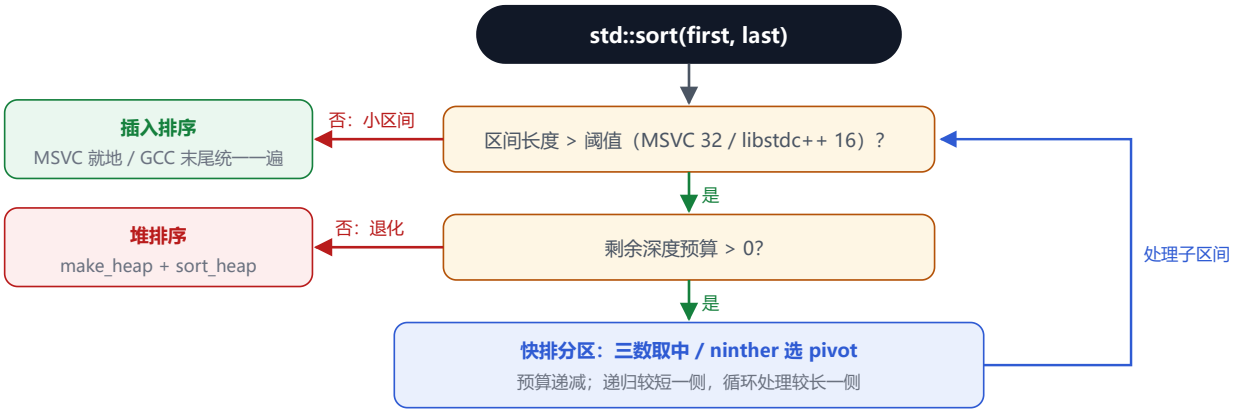


图 8-1　introsort 的控制流：三种算法各自负责一种情况

实现	小区间阈值	pivot	深度预算	插排时机
libstdc++（GCC）	16	三数取中（取 first+1、mid、last−1，放到 first）	$2\lfloor \log_2 n \rfloor$	递归时跳过 ≤ 16 的区间，最后对整个数组做一遍插入排序：前 16 个有边界检查，之后**无边界检查**
MSVC STL	32（`_ISORT_MAX`）	区间 > 40 用 ninther，否则三数取中	每次分区 `_Ideal = _Ideal/2 + _Ideal/4`	每个小区间就地插入排序
libc++（LLVM）	按类型调整	三数 / 九数取中	较新版本加入了深度限制	对算术类型使用无分支的块分区（BlockQuicksort 思路）

深入一层：MSVC 分区返回的是一个“相等区间”

MSVC 的 `_Partition_by_median_guess_unchecked` 返回一对迭代器 `[mid_first, mid_last)`，表示所有等于 pivot 的元素已经聚到中间，下一步只需要递归两侧。这相当于内置了三路划分，所以 E5 里大量重复（16 种值）的数据，`std::sort` 只要 9.68 ms，比本书的 Hoare 工程版（16.29 ms）快很多。

实测 E5：qsort 为什么比 std::sort 慢 66%

100 万个随机 int：`qsort` 85.29 ms，`std::sort` 51.38 ms。`qsort` 的比较器通过函数指针、以 `const void*` 参数调用，没法内联，每次比较都是一次间接调用；`std::sort` 的比较器是模板参数，lambda 会被直接内联成一条 `cmp` 指令。这也是“C++ 模板比 C 回调更快”的经典例证。

8.2 STL 排序接口怎么选

接口	算法	复杂度	稳定	额外内存	典型用途
`std::sort`	introsort	O(n log n)	✗	O(log n) 栈	默认选择
`std::stable_sort`	归并（缓冲不够时用原地归并）	有缓冲 O(n log n)，否则 O(n log² n)	✓	**尝试分配** O(n)	UI 层级、多关键字
`std::partial_sort`	堆选择 + sort_heap	O(n log K)	✗	O(1)	排行榜前 K 名
`std::nth_element`	introselect	平均 O(n)	✗	O(1)	中位数、第 K 近的敌人
`std::partition`	单次分区	O(n)	✗	O(1)	把存活单位挪到前面
`std::list::sort`	归并	O(n log n)	✓	O(1)	链表
`std::ranges::sort`	同 `std::sort`	同上	✗	同上	C++20，支持投影：`ranges::sort(v, {}, &Hero::hp)`

`std::sort(std::execution::par, ...)`（C++17）可以多线程排序。在游戏客户端里一般不这么用，引擎通常有自己的任务系统，更常见的做法是在 Job 里把数据分块排序再归并，或者干脆把排序放到 GPU 上。

Top-K：为什么 partial_sort 比 nth_element 还快

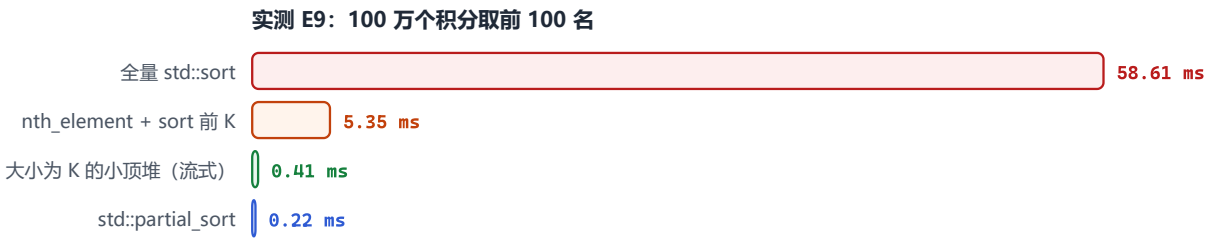


图 8-2　K 远小于 n 时，堆方法几乎只是把数据扫描一遍

直觉上 `nth_element` 是 O(n)，`partial_sort` 是 O(n log K)，前者应该更快。但实测反过来了，因为 K = 100 远小于 n：堆里装着当前前 100 名，绝大多数新元素都比堆顶小，**只比较一次就被丢掉**，只有极少数需要入堆调整。实际工作量接近 n 次比较。`nth_element` 则要做好几轮完整的分区，每轮都有大量交换。**K 很小用堆，K 和 n 同量级用 nth_element。**

8.3 严格弱序：比较器必须遵守的契约

标准要求比较器 `comp` 是**严格弱序**（strict weak ordering），即满足：

性质	要求	写成 `≤` 会怎样
非自反	`comp(a, a)` 必须为 false	**违反**：`a ≤ a` 为 true
非对称	`comp(a, b)` 为 true，则 `comp(b, a)` 为 false	**违反**：a == b 时两个方向都为 true
传递	`comp(a, b)` 且 `comp(b, c)`，则 `comp(a, c)`	成立
不可比的传递性	a~b 且 b~c，则 a~c（~ 表示“谁也不小于谁”）	—

违反这个契约是**未定义行为**。具体后果取决于实现，下面是实测结果：

```
E7 · 比较器写成 ≤（64 个全相等的元素，数组左边是别人的内存 [0,9,9,9,9,9,9,9]）

(a) libstdc++ 式无哨兵插排 + ≤：越界读 336 次；排序后数组里出现了 7 个从未出现过的 9
    左侧内存被改成 [0,5,5,5,5,5,5,5]
    同一段代码改用 < ：越界读 0 次，左侧内存未被改写
(b) MSVC std::sort + ≤（2000 个相等元素，Release）：比较 14635 次，访问哨兵区 0 次
```

```
同样数据用 < ：比较 4006 次
（MSVC Debug 版会通过 _DEBUG_LT_PRED 断言 "invalid comparator"）
```

原理：libstdc++ 的收尾插入排序在前 16 个元素之后不检查边界，它假设“前 16 个里一定有一个不大于 key 的元素能挡住它”。用 `≤` 时，相等也算“小于”，key 会一直往左走，越过数组开头，读到别人的内存；后移操作又把别人的数据挪了进来、把 5 写了出去。**这就是“比较器写错导致崩溃，而且崩溃点离 sort 很远”的来源。**MSVC 的实现在这个例子里没有越界，但比较次数多了 3.6 倍；Debug 版会直接断言。不同实现表现不同，正说明这是 UB，不能依赖。

常见的比较器错误

比较器：错误写法与正确写法

```
struct Hero { int rank; float winRate; int id; };

// ✗ 1：用 ≤ 或 ≥
auto bad1 = [](const Hero& a, const Hero& b) { return a.rank ≤ b.rank; };

// ✗ 2：多字段写成“或”：a=(1,9) b=(2,1) 时 bad2(a,b) 和 bad2(b,a) 都为 true
auto bad2 = [](const Hero& a, const Hero& b) { return a.rank < b.rank || a.winRate < b.winRate

// ✗ 3：浮点里有 NaN：NaN 与任何数比较都为 false，破坏了“不可比”的传递性
// ✗ 4：比较器依赖会变化的外部状态（排序过程中修改了 winRate、读了随机数、读了实时时间）

// ✓ 字典序：用 std::tie 最不容易写错
auto good = [](const Hero& a, const Hero& b) {
    return std::tie(a.rank, a.winRate, a.id) < std::tie(b.rank, b.winRate, b.id);
};
// ✓ 降序就交换 a、b，或者用 >：两者都是严格弱序
auto desc = [](const Hero& a, const Hero& b) { return a.rank > b.rank; };
// ✓ C++20：默认生成 ⟺，直接 std::sort(v.begin(), v.end())
// struct Hero { ... ; auto operator⟺(const Hero&) const = default; };
```

游戏客户端视角：Lua 和 C# 里也有同样的坑

- **Lua** `table.sort`：快排实现，不稳定。比较函数写成 `a.lv ≤ b.lv` 时会报 `invalid order function for sorting`，或者在数组边界外读到 nil 而报错；数据不同，有时报错有时不报，线上很难排查。
- **C#** `List<T>.Sort` / `Array.Sort`（Unity 常用）：introsort，**不稳定**。比较器不一致时，.NET 可能抛出 “IComparer.Compare() method returns inconsistent results”。要稳定排序用 LINQ 的 `OrderBy`（稳定，但会分配内存）。

8.4 现代排序：pdqsort 和 TimSort

pdqsort（Orson Peters，2016）

Pattern-defeating quicksort，introsort 的改进版：

· 检测到分区很不平衡时，打乱几个元素来破坏对手构造的模式

· 分区时没有发生交换，就尝试有上限的插入排序，有序输入接近 O(n)

· 块分区（BlockQuicksort）：先把比较结果存进偏移数组，再批量交换，消除分支预测失败

Go 1.19 的 `sort`、Boost 的 `pdqsort` 都采用了它；Rust 的 `sort_unstable` 也长期基于 pdqsort。

TimSort（Tim Peters，2002）

稳定的归并 + 插入混合排序，针对真实数据中常见的“有序段”：

· 扫描找出天然的升序段或降序段（降序段直接翻转），短段用二分插入补齐到 minrun（32 ~ 64）

· 用栈维护各段长度的不变式，决定何时合并

· galloping 模式：一边连续获胜时，用指数搜索成块拷贝

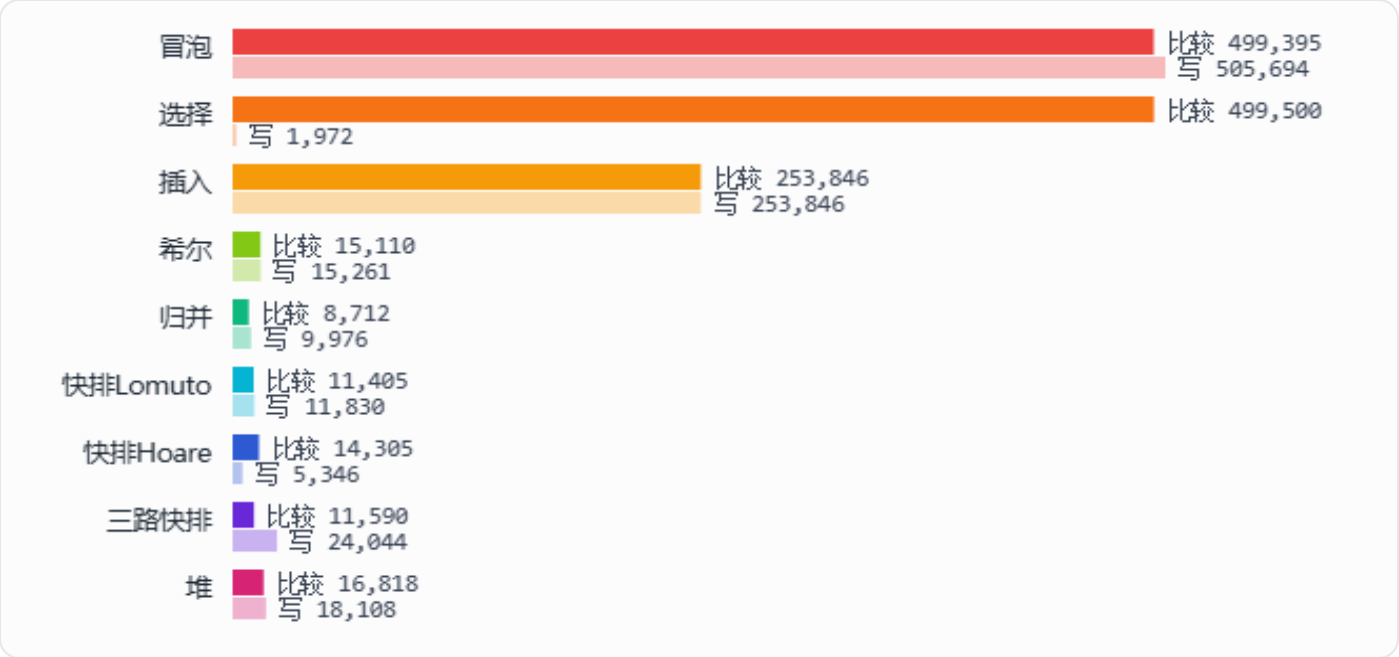
Python 的 `sorted`、Java 的对象数组排序、Android 都使用它。有序输入 O(n)，最坏 O(n log n)。

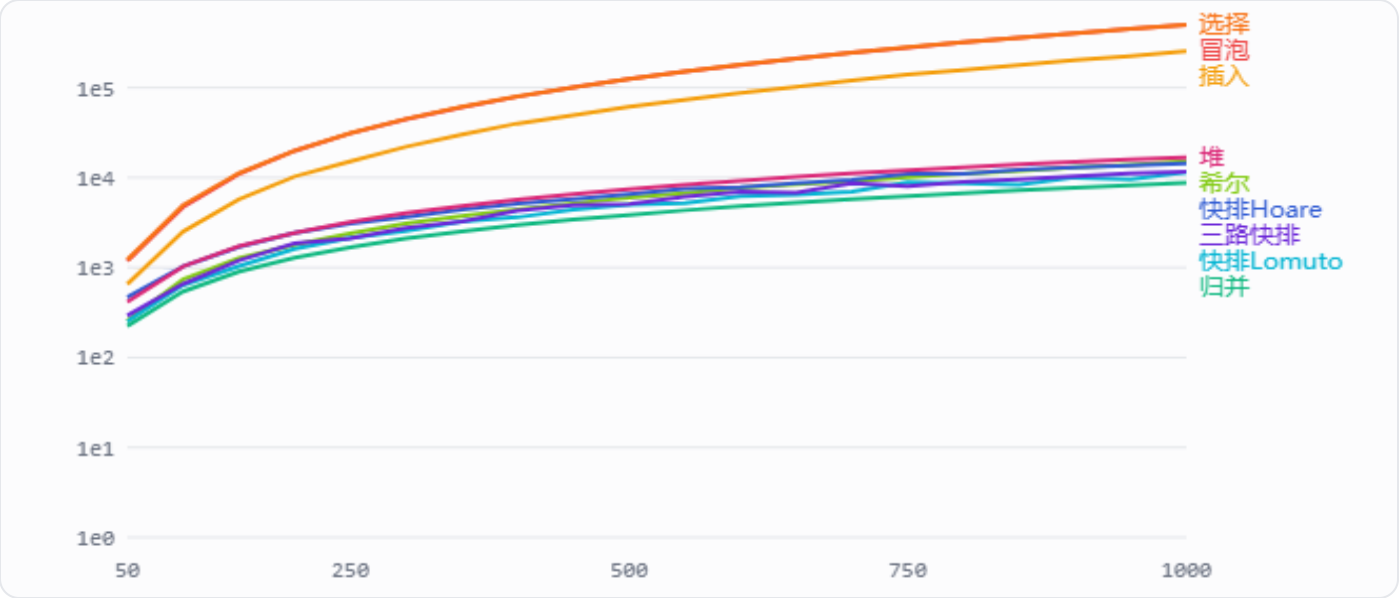
另外，Java 对基本类型数组用的是 Dual-Pivot Quicksort（Yaroslavskiy，2009）：用两个 pivot 把数组分成三段，减少了内存访问次数。

8.5 交互实验：九种比较排序同场竞技

选不同的数据分布，看柱状图里各算法的比较次数和写入次数；下方的对数坐标曲线显示比较次数随 N 增长的趋势：$O(n^2)$ 的线斜率明显更陡。试试“已有序”下的快排 Lomuto 和冒泡。

交互实验 · 九种比较排序同场竞技（只计数，不播动画）





面试注意

- 回答“std::sort 是怎么实现的”，至少要说出：**introsort；快排主体 + 三数取中；深度超过约 2 log n 转堆排；小区间（16/32）用插入排序；不稳定；C++11 起要求最坏 O(n log n)。**
- 被问“std::sort 需要随机访问迭代器吗”：需要。所以 `std::list` 不能用 `std::sort`，只能用成员函数 `list::sort`。
- `stable_sort` 会尝试分配 O(n) 的临时内存，在每帧都要执行的代码里要注意。

本章速记

- introsort = 快排 + 堆排兜底 + 插排收尾；阈值 libstdc++ 16 / MSVC 32；深度预算约 $2\log_2 n$。
- qsort 慢在比较器无法内联（E5：85 ms 对 51 ms）。
- K 很小时 `partial_sort` / 堆最快（E9：0.22 ms），K 较大时用 `nth_element`。
- 严格弱序：非自反、非对称、传递、不可比传递；写成 `≤` 是 UB，可能越界写（E7）。用 `std::tie` 写多字段比较。

CHAPTER 09

实测大比拼：复杂度之外的真相

同一台机器、同一个编译器、同一份数据，13 种实现、5 种分布。看数据，比背结论更有说服力。

测试环境

Windows 11 x64，MSVC 14.44（`cl /std:c++latest /EHsc /O2`），int 数组，每项取 3 次中最快的一次。数据分布：随机（0 ~ 2^{31}）、已有序、逆序、近乎有序（随机交换 1% 的位置）、大量重复（只有 16 种值）。源码 `demos/sort_demo.cpp`，原始输出 `demos/run_output.txt`。

9.1 N = 20,000：O(n²) 组与 O(n log n) 组的差距

E3（ms）	随机	已有序	逆序	近乎有序	大量重复
冒泡	75.01	**0.01**	72.68	77.45	73.73
选择	**282.86**	**280.08**	**302.03**	**299.99**	**298.11**
插入	20.57	**0.01**	40.00	0.57	18.51
希尔	1.19	0.15	0.11	0.71	0.46
归并	0.98	0.06	0.22	0.18	0.58
快排-朴素（Lomuto，末元素）	0.71	**61.26**	**57.41**	0.56	2.86
快排-工程（三数取中 + Hoare）	0.65	0.06	0.06	0.08	0.29
快排-三路（随机 pivot）	0.68	0.23	0.24	0.41	0.17
堆	1.00	0.65	0.69	0.57	0.74
std::sort	0.71	0.08	0.10	0.08	0.19

两个数量级的差距：随机数据下，最快的 O(n²)（插入，20.57 ms）比最慢的 O(n log n)（堆，1.00 ms）还慢 20 倍。**只要 n 上了万，就别再考虑 O(n²) 算法**，除非数据近乎有序（插入 0.57 ms）。

9.2 N = 1,000,000：O(n log n) 组内部的较量

E5（ms）	随机	已有序	逆序	近乎有序	大量重复
希尔	87.67	5.88	8.02	62.86	23.81
归并	70.07	3.20	15.48	13.44	34.51
快排-工程	47.27	3.94	4.08	5.23	16.29
快排-三路	49.34	20.84	16.57	18.47	**7.95**
堆	73.59	**35.90**	**36.09**	**29.61**	34.81
基数（LSD，8 bit × 4 趟）	**4.31**	5.97	5.78	6.05	**2.60**
std::sort	51.38	3.91	5.03	5.86	9.68
std::stable_sort	51.65	4.48	7.25	8.16	18.98
qsort	85.29	22.76	23.47	24.95	17.26

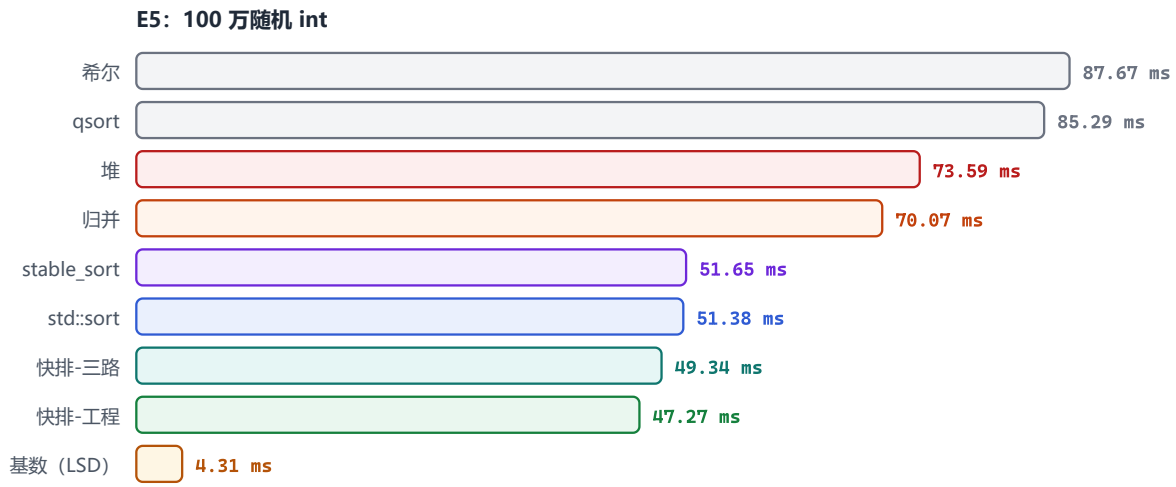


图 9-1　比较排序都挤在 47 ~ 88 ms 之间，基数排序快了一个数量级

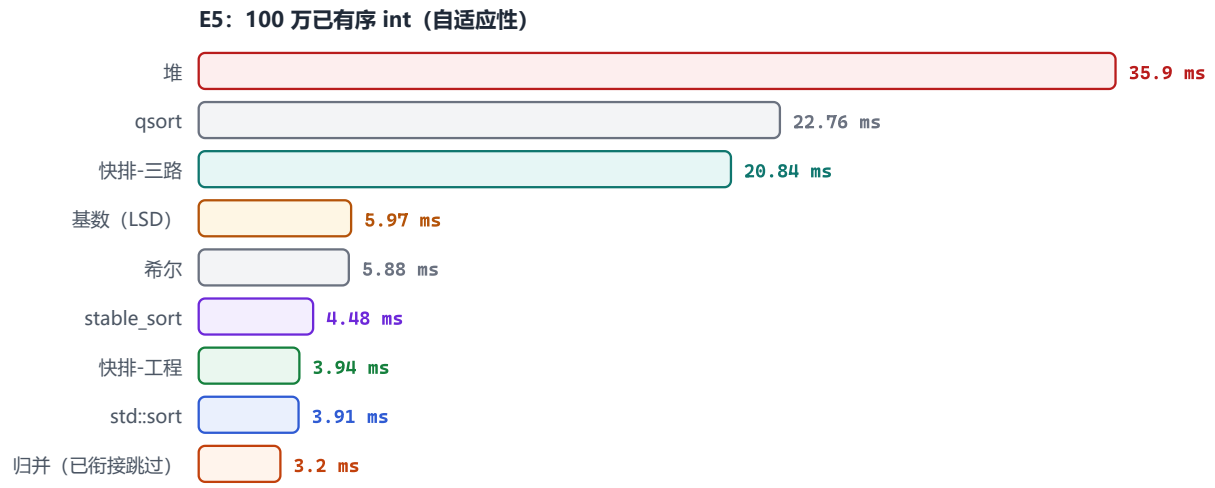


图 9-2　有序输入下排名几乎反转：堆排和基数排序不会因为数据有序而变快

9.3 操作计数：比较次数并不决定一切

E2（N=1000）	随机：比较	随机：写	有序：比较	有序：写	逆序：比较	逆序：写
冒泡	496,260	483,422	999	0	499,500	999,000
选择	499,500	**1,982**	499,500	0	499,500	1,000
插入	242,702	242,710	999	999	499,500	500,499
希尔	12,960	13,400	6,771	6,771	9,354	9,861
归并	**9,401**	18,686	999	0	6,043	19,952
快排-朴素	10,453	12,568	**499,500**	**1,000,998**	**499,500**	500,998
快排-工程	11,171	6,730	7,188	936	7,189	1,940
快排-三路	15,936	20,188	17,058	21,505	16,348	20,337
堆	16,840	11,585	17,583	12,206	15,965	10,814

归并的比较次数最少（9401，低于 $n \log_2 n \approx 9966$），但实测比快排慢，因为它的写入次数是快排的 2.8 倍，而且要来回搬运缓冲区。堆排的比较次数和写入次数都不算夸张，却是 O(n log n) 组里最慢的，代价藏在缓存未命中里，计数根本看不出来。

9.4 从数据中得出的十条结论

1. **默认用 `std::sort`**：各种分布下都在前列，没有明显短板。
2. **快排的平均速度确实最快**（47.27 ms），但“朴素版”遇到有序数据慢了 1000 倍（61.26 ms 对 0.06 ms），还可能栈溢出。
3. **堆排最坏有保证，但实际最慢**，而且完全不自适应。
4. **基数排序在大数据上碾压比较排序**（12 倍），但要额外内存，有序时也不占优。
5. **三路快排只在重复多时有优势**（7.95 ms），没有重复时比 Hoare 慢。
6. **插入排序对近乎有序的数据有奇效**（2 万个数 0.57 ms，和 O(n log n) 算法同一水平）。
7. **stable_sort 的代价不大**：随机数据下与 std::sort 持平（51.65 对 51.38 ms），但会申请内存。
8. **qsort 比 std::sort 慢 66%**，原因是比较器无法内联。
9. **$O(n^2)$ 的常数可以相差十几倍**，与写法、代码生成密切相关。
10. **比较次数 ≠ 耗时**：写入次数、访存模式、分支预测同样重要。

面试注意：用数据说话，但要说明条件

引用实测数据时，带上规模、数据类型和编译选项，例如“在 100 万个 int、MSVC /O2 下，基数排序大约比 std::sort 快 12 倍”。不要说“基数排序比快排快 12 倍”，换成大结构体、换成 1000 个元素，结论都会变。

PART SIX

第六部分　游戏客户端里的排序

面试官问完原理，往往会接一句“你在项目里怎么用排序”。这一部分给出六个真实场景：每个场景里排序都不只是“调一下 std::sort”。

CHAPTER 10

六个实战场景

渲染队列、透明物体、UI 层级、帧同步、排行榜、碰撞粗检测，再加上小栈线程和内存分配这两条工程约束。

本章目标

- 能设计一个 64 位渲染 sort key，并解释每个字段的排序意图
- 能说清帧同步里排序导致不同步的原因和解决办法
- 知道帧间相干性（数据在帧与帧之间变化很小）怎么用于加速排序

10.1 渲染队列：64 位 sort key + 基数排序

每帧要提交几千到几万个 draw call。提交顺序会影响三件事：

- **状态切换开销**：切换 shader、材质、纹理都很贵。相同材质的物体排在一起，就能少切换几次。
- **Overdraw**：不透明物体**从近到远**绘制，后画的远处像素会被深度测试（Early-Z）提前剔除，省掉片元着色。
- **正确性**：半透明物体必须**从远到近**绘制，否则混合结果是错的。

这几个目标有优先级：先分渲染层（背景、世界、特效、UI），层内先不透明后透明；不透明物体优先按材质归并，透明物体只能按深度。经典做法是把这些字段**按优先级从高到低压进一个 64 位整数**，只对这个整数排序：

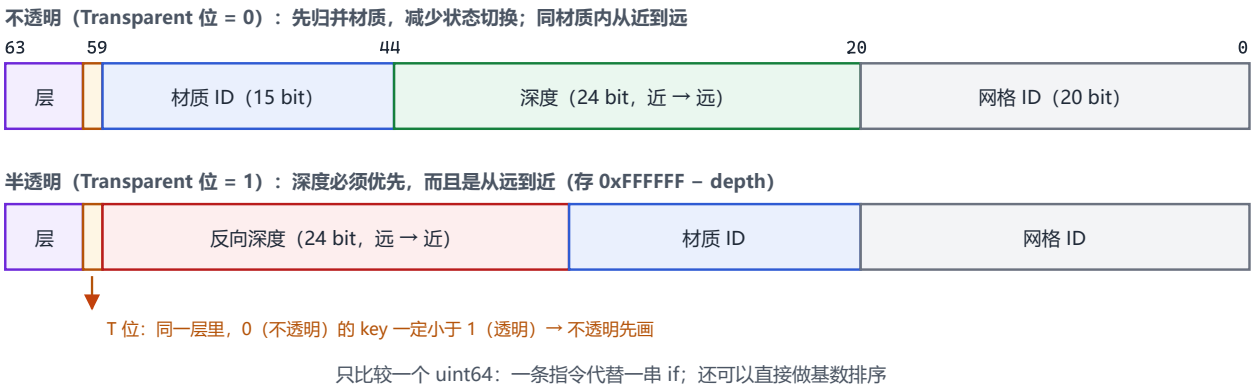


图 10-1 渲染 sort key 的位布局（与 demo 中的 `make_key` 一致）：高位决定大顺序，低位决定小顺序

make_key：把排序意图编码进整数

```
uint64_t make_key(uint32_t layer, bool transparent, float depth01, uint32_t mat, uint32_t mesh
    uint32_t d = uint32_t(depth01 * 0xFFFFFF);            // 深度量化到 24 bit（depth01 ∈ [0,1
    uint64_t k = (uint64_t(layer & 0xF) << 60) | (uint64_t(transparent) << 59);
    if (transparent)
        k |= (uint64_t(0xFFFFFF - d) << 35) | (uint64_t(mat & 0x7FFF) << 20) | (mesh & 0xFFFFF
    else
        k |= (uint64_t(mat & 0x7FFF) << 44) | (uint64_t(d) << 20) | (mesh & 0xFFFFF);
    return k;
}

struct SortItem { uint64_t key; uint32_t index; };      // 16 字节，只排它，不排 64 字节的 DrawI
// 每帧：items[i] = { make_key( ... ), i }；radix_sort_u64(items, tmp, n, [](auto& x){ return x.k
// 提交：for (auto& it : items) submit(drawList[it.index]);
```

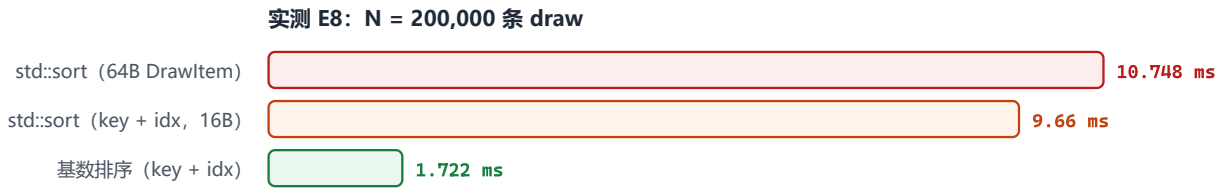


图 10-2 20 万条 draw：基数排序比 std::sort 快约 5.6 倍；N=2 万时是 0.151 ms 对 0.761 ms；N=2000 时三者都在 0.03 ms 以内

- **只排 (key, index)，不排大结构体**：交换 16 字节比交换 64 字节便宜。本例中 DrawItem 只有 64 字节，差距约 10%；结构体越大、拷贝越贵（比如带 `std::string`），差距越大。

- **基数排序非常适合 sort key**：键定长、数据量大、每帧都要排。缓冲区用帧分配器（frame allocator）每帧复用，不要每帧 new。高位字节经常全部相同（比如所有物体都在同一层），可以跳过这些趟，demo 里就是这样做的。
- **深度只量化成 24 bit** 是有意为之：远处物体的精确先后对 Early-Z 影响很小，省下的位可以留给材质。
- 引擎里的对应物：UE 的 `FMeshDrawCommand` 带有排序键，Unity 按 Sorting Layer、Order in Layer、Render Queue、距离等依次决定顺序，思路是一致的。

10.2 透明物体与粒子：排序只能解决一部分

- **按物体排序（画家算法）**只能保证物体之间的先后。两个相互穿插的透明物体，无论谁先画都会有一部分是错的；要彻底解决得用 OIT（顺序无关透明），或者在美术上避免穿插。
- 排序依据一般用“包围盒中心到相机的距离平方”，不需要开方，因为平方保持大小顺序。
- 大规模粒子在 GPU 上排序：双调排序（bitonic sort，O(n log² n)，但比较的位置是固定的，适合大规模并行）或 GPU 基数排序。数量大但精度要求不高时，也可以用第 7 章的深度分桶做近似排序。
- **帧间抖动**：两个透明物体距离很接近时，相机稍微一动，它们的顺序就来回交换，画面出现闪烁。可以给距离加一个滞后阈值，或者用稳定的次级键（如实体 id）决出平局。

10.3 UI 层级：稳定性决定遮挡关系

UI 节点先按 layer（或 sortingOrder）排，**同一层内必须保持创建或层级顺序**，后创建的盖在上面。用不稳定排序时，同层节点的先后会被打乱，表现为弹窗被背景盖住、点击穿透、每次打开顺序不一样。

```
E6 · 64 个 UI 节点，layer 只有 0/1/2，按 layer 排序

std::sort     layer0: N00 N21 N57 N03 N30 N54 N06 N39 N24 N09 N33 N51 ...
stable_sort   layer0: N00 N03 N06 N09 N12 N15 N18 N21 N24 N27 N30 N33 ...
两种结果有 57 / 64 个位置不同（同层遮挡关系被打乱 → UI 闪烁 / 层级错乱）
```

两种修法：①用 `std::stable_sort`；②把原始顺序放进键里，用 `std::tie(layer, siblingIndex)` 比较，这样任何排序算法都会得到同一结果。第二种更好：结果不依赖算法是否稳定，也避开了 `stable_sort` 的内存分配。

面试注意：元素少时 std::sort 也“看起来稳定”

元素个数不超过插入排序阈值（MSVC 32、libstdc++ 16）时，`std::sort` 实际上只做了一遍插入排序，结果恰好是稳定的。所以测试时 10 个节点一切正常，上线后 100 个节点就乱了。E6 特意用了 64 个节点。**“这次碰巧稳定”不等于“保证稳定”。**

10.4 帧同步：平局顺序必须在所有客户端一致

帧同步（lockstep）的 MOBA 中，所有客户端只交换玩家输入，各自运行同一套逻辑，要求每帧的计算结果**逐位一致**。如果某段逻辑“按威胁值排序，攻击第一个敌人”，而两个敌人的威胁值相等，那么谁排第一取决于排序算法如何处理平局：

```
E10 · 2000 个单位按威胁值排序（威胁值只有 10 档，大量平局）

只比 score（有平局）：
  std::sort (MSVC)          hash=0867a2906420d9bb  首个目标 id=1115
  快排-工程 (另一种实现)     hash=a9fd6e705baa4d93  首个目标 id=1228
  堆排序                    hash=1dddafa554ba0db7  首个目标 id=1161
  stable_sort               hash=32655e286d26c10b  首个目标 id=1
score 相同再比 id（全序）：
  std::sort / 快排-工程 / 堆排序 / stable_sort   hash 全部 = 32655e286d26c10b  首个目标 id=1
```



图 10-3 同一份输入、同一行代码，在两个平台上选出了不同的目标

帧同步排序检查清单

1. **比较器必须是全序**：最后一个比较字段用逻辑 id（网络 id / 创建序号）兜底，保证不存在“谁也不小于谁”的两个元素。

2. **不要用 Entity 的内存地址或运行时句柄排序**：地址受 ASLR 和分配顺序影响，各端不同；客户端独有的实体（特效、本地 UI 实体）会让槽位编号错开。
3. **不要遍历哈希容器的顺序**：`std::unordered_map` 的遍历顺序取决于哈希函数、桶数和插入历史，不同 STL 实现各不相同。要么先收集到数组里按 id 排序，要么用有序容器。
4. **排序键里不要有浮点运算的结果**：距离等浮点值在不同编译器、不同指令集下可能有末位差异，平局判断会不同。帧同步逻辑一般使用定点数。
5. **想要绝对保险，就各端用同一份自己实现的排序**（例如一份固定的归并排序），不依赖平台 STL。

> **游戏客户端视角：这类不同步最难查**
>
> 平局只在特定数据下出现，两端大多数时候结果一致，只有偶尔一局在某一帧分叉。排查时要在每帧对关键状态做哈希，不一致时把前后几帧的“排序输入 + 输出”都 dump 出来对比。本系列 Q04（ECS 怎么 debug）里“按 NetId 决胜”的修复思路，解决的也是同一类问题。

10.5 排行榜与 Top-K

- **只显示前 K 名**：用 `partial_sort` 或大小为 K 的堆（E9：100 万人取前 100 名，0.22 ms / 0.41 ms，全量排序 58.61 ms）。
- **只要第 K 名的分数线**（例如“前 10% 的分数线”）：用 `nth_element`，平均 O(n)。
- **单个玩家分数变化**：不必整体重排。在有序数组里删掉旧位置、二分找到新位置插入，O(log n) 查找 + O(n) 移动；如果改动很频繁，就换成平衡树、跳表这类有序结构（服务端常用 Redis 的 ZSet，底层是跳表）。
- **好友榜**：好友数量一般不超过几百，直接 `std::sort` 就行。这时可读性比优化更重要。

前 K 名（降序，名次并列时按 id）

```
struct Entry { int score; int id; };
void top_k(std::vector<Entry>& v, size_t k) {
    k = std::min(k, v.size());
    std::partial_sort(v.begin(), v.begin() + k, v.end(), [](const Entry& a, const Entry& b) {
        return a.score != b.score ? a.score > b.score : a.id < b.id;   // 全序：名次稳定可复现
    });
    // v[0..k) 就是前 k 名，且已经有序；v[k..) 的顺序不确定
}
```

10.6 Sweep and Prune：利用帧间相干性

碰撞检测的粗检测阶段要找出“包围盒可能相交的物体对”。Sweep and Prune（SAP）把每个物体的 AABB 投影到 x 轴上，得到一个区间 [min, max]。**把所有端点排序**，然后从左到右扫一遍：遇到 min 就把物体加入“活动集”，遇到 max 就移出；加入时与活动集里的每个物体都构成候选对。

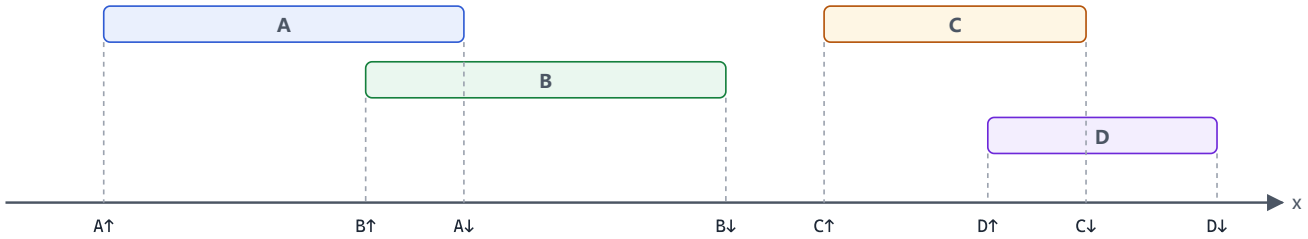


图 10-4 Sweep and Prune：端点排序是核心，帧间相干性让增量排序几乎免费

物体在一帧内只移动很小的距离，所以上一帧排好的端点序列这一帧几乎仍然有序。用插入排序增量更新，代价约为 O(n + 交换次数)，而且**每发生一次相邻交换，就对应一对区间开始或停止重叠**，可以顺便增删候选对。Bullet 物理引擎的 `btAxisSweep3` 就是这样增量维护的。这与 E8 的结论一致：近乎有序时插入排序 0.005 ms，`std::sort` 0.062 ms。

注意它的退化

大量物体同时剧烈运动（爆炸、传送、场景切换）时，相干性没有了，插入排序会退化成 $O(n^2)$。稳妥的做法是：如果一帧的交换次数超过阈值，或者检测到大规模传送，就改用 `std::sort` 整体重排。

10.7 两条工程约束：栈和内存

小栈线程不能深递归

E4：朴素快排 + 有序输入，64 KB 栈大约 500 层就溢出，1 MB 栈 2 万个元素就溢出。Job 线程、纤程、协程的栈往往只有几十 KB。

对策：用 `std::sort`（深度有限）；自己实现时递归短边、加深度限制，或者改成迭代。

每帧分配是性能杀手

`std::stable_sort` 会尝试分配 O(n) 的临时缓冲；归并和基数排序都需要 O(n) 缓冲。

对策：缓冲区放在帧分配器或成员变量里复用；或者用“键 + 原始下标”的全序比较，配合 `std::sort` 得到确定的结果。

本章速记

- 渲染队列：64 位 sort key（层 → 透明 → 材质 / 深度 → 网格）；只排 (key, idx)；基数排序（E8：20 万条 1.72 ms 对 9.66 ms）。
- 透明物体从远到近，不透明物体从近到远；穿插问题排序解决不了。
- UI 同层必须稳定：`stable_sort` 或在键里加 siblingIndex；n ≤ 16/32 时 std::sort 碰巧稳定，这是陷阱。
- 帧同步：比较器必须是带 id 兜底的全序；不按地址排序，不依赖哈希容器的遍历顺序。
- Top-K 用 partial_sort / 堆；SAP 用插入排序做增量更新，剧烈变化时退回整体排序。

APPENDIX

附录　面试题库与速记卡

40 道追问，按“基础 → 算法细节 → STL 与 C++ → 游戏场景 → 手写代码”排列。先自己口头回答，再展开看参考答案。

CHAPTER 11

40 道面试追问

每道题的参考答案都压缩到能在 30～60 秒内讲完的长度。

11.1 基础概念（1～8）

▶ 1. 什么是稳定排序？举一个需要稳定性的例子。

▶ 2. 哪些排序是稳定的？怎么快速判断？

▶ 3. 比较排序的下界是多少？怎么证明？

▶ 4. 计数排序 O(n)，和下界矛盾吗？

▶ 5. 什么是逆序对？和哪些排序有关？

▶ 6. 什么是自适应排序？哪些算法是自适应的？

▶ 7. 原地排序是什么意思？快排算原地吗？

▶ 8. 为什么小数组用插入排序更快？

11.2 算法细节（9～20）

▶ 9. 快排最坏情况什么时候出现？如何避免？

▶ 10. Lomuto 和 Hoare 分区有什么区别?

▶ 11. Hoare 分区里 pivot 取 a[hi] 会出什么问题?

▶ 12. 快排的空间复杂度是多少?怎么保证 O(log n)?

▶ 13. 三路快排解决了什么问题?代价是什么?

▶ 14. 为什么快排实际上比堆排快?

▶ 15. 自底向上建堆为什么是 O(n)?

▶ 16. 升序堆排序用大顶堆还是小顶堆?

▶ 17. 归并排序的空间复杂度?能原地吗?

▶ 18. 链表排序用什么算法?为什么?

▶ 19. 基数排序为什么从低位开始?每趟为什么要稳定?

▶ 20. 浮点数怎么做基数排序?

11.3 STL 与 C++ (21 ~ 30)

▶ 21. std::sort 是怎么实现的?

▶ 22. std::sort 能排 std::list 吗?

▶ 23. std::stable_sort 的实现和代价?

▶ 24. partial_sort 和 nth_element 有什么区别？怎么选？

▶ 25. 什么是严格弱序？比较器写 <= 会怎样？

▶ 26. 多字段比较器怎么写才不会错？

▶ 27. qsort 为什么比 std::sort 慢？

▶ 28. 排序大结构体时怎么优化？

▶ 29. 什么是 pdqsort？什么是 TimSort？

▶ 30. C++20 ranges::sort 的投影是什么？

11.4 游戏场景（31～37）

▶ 31. 渲染队列怎么排序？

▶ 32. 帧同步游戏里排序要注意什么？

▶ 33. UI 层级排序为什么必须稳定？为什么测试时没发现问题？

▶ 34. 100 万玩家取前 100 名，怎么做？

▶ 35. 每帧变化很小的数据怎么排？

▶ 36. 在 Job 线程里手写快排会有什么风险？

▶ 37. Lua 和 C# 里的排序有什么坑？

11.5 手写代码（38～40）

▶ **38. 手写快排（5 分钟版）**

▶ **39. 手写堆排 / 求第 K 大**

▶ **40. 手写链表排序 / 合并 K 个有序链表**

11.6 速记卡

一页纸背完排序

- **O(n²)**：冒泡（稳，带标志位最好 O(n)）、选择（不稳，交换 ≤ n−1，不自适应）、插入（稳，O(n + 逆序对)，小区间之王）。
- **希尔**：分组插入，Knuth / Hibbard 增量 O(n^1.5)，不稳定。
- **归并**：稳定，三种情况都是 n log n，O(n) 空间；链表、外排、逆序对。
- **快排**：平均最快，最坏 n²，栈 O(log n)；Lomuto 好写、Hoare 交换少、三路治重复；工程四件套：取中、Hoare、插排收尾、递归短边。
- **堆排**：最坏 n log n，O(1) 空间，不稳定，缓存差；建堆 O(n)；升序用大顶堆。
- **非比较**：计数 O(n+k)、基数 O(d(n+b)) 每趟必须稳定、桶排序均匀分布期望 O(n)。
- **STL**：sort = introsort（16/32、2 log n、堆兜底）；stable_sort = 归并（会分配）；partial_sort = 堆；nth_element = introselect；比较器必须是严格弱序。
- **游戏**：64 位 sort key + 基数排序；UI 用稳定排序；帧同步用 id 兜底的全序；Top-K 用堆；帧间相干用插入排序；小栈线程不能深递归。